

Numerical propagation of hyperfine-state populations through a Sona transition unit: status of the simulation code

Sona transition simulation project
based on the theoretical framework of Chapters 3 and 6, Kannis (2023)

September 29, 2026

Abstract

This note summarizes the current status of the Python code being developed to propagate the hyperfine-substate population of H/D atoms through a Sona transition unit, using magnetic field maps of the RHIC OPPIS as measured or simulated input. The formalism and code apply identically to **both** the ground ($1S_{1/2}$) state – the atoms Sona transitions actually act on in RHIC OPPIS operation – and the metastable ($2S_{1/2}$) state used in the Lamb-shift-polarimeter bench measurements (Sec. 2.5) that the code is validated against; the two cases differ only in the numerical value of the hyperfine constant A (Table 2.1). The full quantum-mechanical target model is the time-dependent perturbation theory formalism of Ch. 6 of the Kannis (2023) thesis [1]. **Update since the previous version of this note:** the coupled-state population propagation itself – construction and diagonalization of $H_0 + V(r, t)$, Eq. (4), and numerical integration of Eq. (5) for a single atom and for a Gaussian-averaged beam – is now implemented and validated (Sec. 2.5), alongside the field-map processing and kinematic estimates that were already in place. The zero-crossing/non-adiabaticity condition (Sec. 2.2) and the identification of the two on-axis mechanisms (Branches 1 and 2, Sec. 2.5) that combine to produce full electron-to-nuclear polarization transfer have also been sharpened, using additional literature (Antishchev & Belov 2008 [4]) and a first-principles conservation-law argument, since the previous version. The current-sweep (polarization vs. Sona-coil current) validation against the thesis’s published curves is the next and last step before the PSTP contribution. This document is intended as a progress update for the group ahead of the contribution to PSTP (Mito, Japan).

Contents

1	Context and goal	1
2	Theoretical framework (Ch. 3 and Ch. 6 of the thesis)	2
2.1	Hyperfine structure in a static field	2
2.2	Zero-crossing (non-adiabaticity) condition	3
2.3	Target dynamical model: time-dependent perturbation theory	3
2.4	Explicit matrix form and physical reading of its elements	4
2.5	Fixed vs. rotating quantization axis, and the two branches of the mechanism	6
2.6	Data ingestion and field-profile extraction	7
2.7	Zero-crossing position and gradient	7
2.8	Adiabaticity check against Sona’s threshold	7
2.9	Effective Sona frequency and energy-scale estimate	8
2.10	Correlation with measured polarization	8
2.11	Interactive inspection tools	8
2.12	Quantum propagation implementation (sona package)	8
2.13	Independent cross-check against a Mathematica implementation	9

3 Plan for further development	11
Appendix: current code listing	11
sona package: quantum propagation	12
Field-map processing: MagScan_Ana4.py	20

1 Context and goal

The Sona transition unit reverses the sign of the longitudinal magnetic field B_z along the beam axis, driving an H (or D) atom prepared in one hyperfine substate through the zero-field region. If the field reversal is “sudden” compared to the Larmor precession, the atom cannot follow the Breit–Rabi energy levels adiabatically, and its wavefunction ends up as a coherent superposition of the substates it started in and the ones it is transferred to on the other side of the crossing – this is the mechanism Sona originally exploited to enhance nuclear polarization. Chapters 3, 5, and 6 of [1] show, however, that even when Sona’s non-adiabaticity condition is satisfied, the measured polarization downstream of the unit oscillates with the current in the Sona coils – a genuinely quantum mechanical effect (coupling between hyperfine substates near the crossing) that a purely semiclassical treatment cannot reproduce.

Ground vs. metastable state – why both appear in this note. In an OPPIS-type source the Sona unit acts on atoms in the *ground* state ($1S_{1/2}$), since that is the atom’s state at the point it is re-ionized to form the polarized H^- (or D^-) beam. The bench measurements used in this project to validate the code, however – the Lamb-shift-polarimeter data of [5] and [1] Ch. 5 – are taken on the *metastable* state ($2S_{1/2}$), which is the experimentally convenient choice for a test bench (it can be selectively quenched and detected via Lyman- α emission; see Sec. 2.1). The Hamiltonian, the FODE system Eq. (5), and the explicit matrix of Sec. 2.4 are *identical in form* for both cases – only the hyperfine constant A (and the derived critical field B_c) differ numerically (Table 2.1). The code is therefore written, and should be read throughout, as applying to either state via a single parameter choice, not as two separate code paths.

The long-term goal of this project is a Python tool that:

1. imports a magnetic field map of the Sona region – either from a simulation (e.g. finite-element solenoid model) or from a measurement (LCCS/NewScanApp bench scans of the OPPIS Sona coils);
2. builds the corresponding $B_z(z)$ and $B_r(z, r)$ profiles seen by an atom travelling through the unit;
3. propagates the hyperfine-substate population amplitudes through that field, following the time-dependent perturbation theory formalism derived in Ch. 6 of [1];
4. compares the resulting polarization vs. Sona-coil-current curve with the measured polarization scans, in place of (or as a generalization of) the Mathematica code used in the thesis.

This tool would let us test new Sona coil geometries and operating points *before* committing beam time to them, and is the basis of the contribution planned for the PSTP conference in Mito.

2 Theoretical framework (Ch. 3 and Ch. 6 of the thesis)

This section recalls – for reference, without re-deriving – the pieces of the theory that the code needs to implement or already uses. Full derivations are in [1].

2.1 Hyperfine structure in a static field

An H or D atom, in either its ground ($1S_{1/2}$) or metastable ($2S_{1/2}$) state, in a field $\mathbf{B} = B\hat{z}$ is described by the same Breit–Rabi Hamiltonian,

$$H = A \mathbf{I} \cdot \mathbf{J} - (g_J \mu_B J_z + g_I \mu_N I_z) B, \quad (1)$$

where $\mathbf{I}$ and $\mathbf{J}$ are the nuclear and electronic angular momenta, μ_B, μ_N are the Bohr and nuclear magnetons, and $g_J \approx 2.0023$, g_I are the electronic and nuclear g -factors (common to both S -states to the precision needed here). The two states differ only through the hyperfine constant A , which is set by the electron probability density at the nucleus, $A \propto |\psi(0)|^2$; since $|\psi_{2S}(0)|^2 = \frac{1}{8} |\psi_{1S}(0)|^2$, the metastable-state constant is (almost exactly) one eighth of the ground-state one:

State	A_H/h	B_c	Source
Ground, $1S_{1/2}$ (RHIC OPPIS operating state)	1420.405 752 MHz	50.7 mT	21 cm line (exact)
Metastable, $2S_{1/2}$ (Lamb-shift-polarimeter bench state)	177.556 860(4) MHz	6.34 mT	[6]

Note on the value of A in the code. `constants.py` currently uses $A_{2S}/h = 177.556$ MHz, the Mathematica reference implementation (Sec. 2.13) uses 177.556 834 3 MHz, and the table above quotes 177.556 860 MHz; the differences change the final populations by $\sim 10^{-4}$ only, but a single value should be adopted everywhere before publication. With the constants of `constants.py` the definition $B_c = A/(|g_J|\mu_B + g_I\mu_N)$ evaluates to 6.326 mT (2S) and 50.61 mT (1S); the quoted 6.34 mT and 50.7 mT correspond to neglecting the nuclear term ($A/|g_J|\mu_B$ gives 6.336 mT and 50.68 mT). (Deuterium values used elsewhere in this note, $B_c = 1.46$ mT, refer to the metastable $2S_{1/2}$ state, matching the bench measurements of Sec. 2.5; the ground-state D critical field scales the same way, by the corresponding A_{2S}/A_{1S} ratio.) Diagonalizing Eq. (1) gives the Breit–Rabi energy levels (α_i, β_i substates) as a function of B , which cross smoothly into the high-field $|m_J, m_I\rangle$ states and connect, at $B = 0$, to the total- F hyperfine states, for either choice of A . The high-field/low-field crossover happens at the corresponding B_c in each case; everywhere else in this note, “high field” and “low field” should be read relative to whichever B_c applies to the state actually being propagated.

One place the two states genuinely differ: state preparation. Upstream of the Sona unit, the atomic beam must first be prepared purely in the α substates (the spinfilter-transmitted states), with the unwanted β population removed. For the *metastable* state this is done via a Stark-induced level crossing: at a suitable field the β sublevels of $2S_{1/2}$ cross the (short-lived) $2P_{1/2}$ sublevels, so an applied electric field quenches them to the ground state via Lyman- α decay, leaving only α population – this is the mechanism described in [5] step (1) and is specific to the metastable state (there is no analogous nearby P -state for $1S_{1/2}$ to Stark-quench into). *Ground-state* OPPIS sources instead rely on a Stern–Gerlach-type sextupole magnet upstream of the Sona unit to spatially separate/select the α states by their magnetic moment. This preparation-stage difference does not affect the Hamiltonian, Eq. (1), or the propagation matrix of Sec. 2.4 – it only sets the initial condition $c_k(t=0)$ that Eq. (5) is integrated from, which in both cases is the same 50/50 α_1, α_2 mixture.

2.2 Zero-crossing (non-adiabaticity) condition

Because of $\nabla \cdot \mathbf{B} = 0$, a longitudinal field reversal is necessarily accompanied by a radial component

$$B_r(z, r) = -\frac{1}{2} \frac{dB_z}{dz} r, \quad (2)$$

so an off-axis atom never actually sees $B = 0$: instead it sees the field direction flip by 180° over a finite time. For the polarization not to be scrambled by the resulting Larmor precession, this flip must be

“sudden”, i.e. faster than the Larmor period. This translates into Sona’s zero-crossing double inequality (thesis Eq. 3.37),

$$\frac{eB_t^2}{2m_e v} \ll \left| \frac{dB_z}{dz} \right| \ll \frac{8m_e v}{er^2} \quad (\text{H}), \quad \frac{eB_t^2}{3m_e v} \ll \left| \frac{dB_z}{dz} \right| \ll \frac{12m_e v}{er^2} \quad (\text{D}), \quad (3)$$

with v the atom velocity, r the beam radius, and B_t a residual transverse stray field. The upper bound in Eq. (3) is already implemented in the code (Sec. 2.5) as an adiabaticity threshold to be checked against the measured field gradient at the crossing.

2.3 Target dynamical model: time-dependent perturbation theory

Sona’s inequalities only guarantee that the transition is non-adiabatic; they do not predict *how* the population redistributes among substates near the crossing, which is the effect responsible for the polarization oscillations observed experimentally. Chapter 6 of [1] addresses this with a fully quantum treatment: in the rest frame of the atom (i.e. using $t = z/v$), the Hamiltonian is split into a static hyperfine part H_0 and a time-dependent Zeeman perturbation $V(r, t)$ built from the measured/simulated $B_z(t)$ and $B_r(r, t)$,

$$H(r, t) = H_0 + V(r, t), \quad H_0 = A \mathbf{I} \cdot \mathbf{J}, \quad V(r, t) = -(g_J \mu_B \mathbf{J} + g_I \mu_N \mathbf{I}) \cdot \mathbf{B}(r, t). \quad (4)$$

Expanding the wavefunction on the eigenbasis $|j\rangle$ of H_0 , $|\psi(r, t)\rangle = \sum_j c_j(r, t) e^{-iE_j t/\hbar} |j\rangle$, and inserting into the Schrödinger equation gives a closed system of coupled first-order differential equations (FODEs) for the amplitudes,

$$i\hbar \dot{c}_k(r, t) = \sum_j c_j(r, t) e^{i\omega_{kj}t} \langle k|V(r, t)|j\rangle, \quad \omega_{kj} = \frac{E_k - E_j}{\hbar}, \quad (5)$$

with k, j running over the $(2J+1)(2I+1)$ hyperfine substates (4 for H, 6 for D). The measurable quantities are the populations $|c_k(r, t)|^2$, integrated over the beam’s radial and velocity distribution.

The eigenbasis $|j\rangle$, made explicit. For hydrogen, $|j\rangle$ is exactly $|1\rangle = |\alpha_1\rangle$, $|2\rangle = |\alpha_2\rangle$, $|3\rangle = |\beta_3\rangle$, $|4\rangle = |\beta_4\rangle$ – not a separate abstract object, but literally the same four states of Sec. 2.1, since $H_0 = A \mathbf{I} \cdot \mathbf{J}$ commutes with F^2 , F_z and its eigenstates are exactly the coupled $|F, m_F\rangle$ combinations:

$$H_0 |1\rangle = \frac{A}{4} |1\rangle, \quad H_0 |2\rangle = \frac{A}{4} |2\rangle, \quad H_0 |3\rangle = \frac{A}{4} |3\rangle, \quad H_0 |4\rangle = -\frac{3A}{4} |4\rangle. \quad (6)$$

This is why the basis is called *coupled*: it is the basis in which H_0 – and only H_0 – is diagonal, in contrast to the *uncoupled* product basis $|m_J, m_I\rangle$, in which H_0 has the off-diagonal $A/2$ flip-flop entry seen in Eq. (7) (Sec. 2.4’s basis-convention note). The implemented code (Sec. 2.12) works in the coupled basis throughout, precisely so that H_0 needs to be diagonalized once (on paper, above) rather than re-diagonalized at every propagation step – each step then only has to build the much smaller $V(B_z, B_r, \phi)$ and add it to the already-known diagonal H_0 .

In the thesis this system is solved numerically in Mathematica (method of lines) for each value of the Sona coil current, and the resulting simulated polarization is compared to the Lamb-shift-polarimeter measurements. **This is the model our own code is meant to reproduce and eventually extend**, using directly the OPPIS field maps rather than an idealized/simulated field shape, and Python/SciPy instead of Mathematica.

2.4 Explicit matrix form and physical reading of its elements

For hydrogen ($I = J = 1/2$) it is worth writing Eq. (4) out explicitly, since the pattern of which matrix elements are zero and which are not *is* the physical content of the Sona mechanism. Using the uncoupled product basis $\{|1\rangle, |2\rangle, |3\rangle, |4\rangle\} = \{|++\rangle, |+-\rangle, |--\rangle, |-+\rangle\}$ (shorthand for $|m_J, m_I\rangle$, i.e. $|1\rangle = \alpha_1$,

$|3\rangle = \beta_3$ are the “stretched” states and $|2\rangle, |4\rangle$ are the high-field asymptotes of α_2, β_4 , direct evaluation of $\mathbf{I} \cdot \mathbf{J} = I_z J_z + \frac{1}{2}(I_+ J_- + I_- J_+)$ and of $\mathbf{J} \cdot \mathbf{B}, \mathbf{I} \cdot \mathbf{B}$ gives

$$H(r, t) = \begin{pmatrix} \frac{A}{4} - \frac{1}{2}(g_J \mu_B + g_I \mu_N) B_z & -\frac{1}{2} g_I \mu_N B_r & 0 & -\frac{1}{2} g_J \mu_B B_r \\ -\frac{1}{2} g_I \mu_N B_r & -\frac{A}{4} - \frac{1}{2}(g_J \mu_B - g_I \mu_N) B_z & -\frac{1}{2} g_J \mu_B B_r & \frac{A}{2} \\ 0 & -\frac{1}{2} g_J \mu_B B_r & \frac{A}{4} + \frac{1}{2}(g_J \mu_B + g_I \mu_N) B_z & -\frac{1}{2} g_I \mu_N B_r \\ -\frac{1}{2} g_J \mu_B B_r & \frac{A}{2} & -\frac{1}{2} g_I \mu_N B_r & -\frac{A}{4} + \frac{1}{2}(g_J \mu_B - g_I \mu_N) B_z \end{pmatrix} \quad (7)$$

in the basis order $(\alpha_1, \alpha_2, \beta_3, \beta_4)$. Every entry has a distinct physical role:

- **Diagonal entries** ($H_{11}, H_{22}, H_{33}, H_{44}$). These are the Breit–Rabi energies: the $\pm A/4$ pieces are the zero-field hyperfine splitting, and the B_z -dependent pieces are the ordinary Zeeman shift. A purely diagonal Hamiltonian *cannot cause any transition* – it only advances each amplitude’s phase, $c_k(t) \rightarrow c_k(0) e^{-i \int E_k dt/\hbar}$, leaving every population $|c_k|^2$ unchanged. Reversing B_z , however fast, never by itself moves population between substates: this is the direct matrix-level statement of why the field reversal alone cannot drive any transition (Sec. 2.5 clarifies what it *does* do).
- **The $H_{24} = H_{42} = A/2$ entry (constant, B -independent).** This is the hyperfine “flip-flop” term $\frac{1}{2}(I_+ J_- + I_- J_+)$, present already at $B = 0$: it is exactly what makes α_2 and β_4 genuine superpositions rather than pure product states at low field, and it is the coupling responsible for the *real, physical* adiabatic transfer of population between them as B_z sweeps through the reversal (the mechanism referred to as “Branch 2” below). Because this term never vanishes, the α_2 – β_4 energy gap $E_2 - E_4 = \sqrt{A^2 + [(g_J \mu_B - g_I \mu_N) B_z]^2}$ never closes (its minimum, at $B_z = 0$, is exactly A), so this pair can be followed adiabatically for essentially any realistic field-sweep rate. **This “constant $A/2$ ” description is specific to the uncoupled basis of Eq. (7).** In the coupled basis used by the implemented code (Eq. (8), $H_{24} = \frac{B_z}{2}(\alpha - \beta)$, which vanishes at $B_z = 0$), the same A -sized gap at $B_z = 0$ instead comes entirely from the *diagonal* splitting ($H_{22} - H_{44} = A/4 - (-3A/4) = A$ there, with zero off-diagonal contribution) – the two bases partition the identical physical gap differently between diagonal and off-diagonal entries, exactly as the basis-convention note above anticipates. The conclusion (a robust, never-closing A -sized gap driving Branch 2) is basis-independent; the specific matrix entry responsible for it is not.
- **The B_r -proportional off-diagonal entries** ($H_{12}, H_{14}, H_{23}, H_{34}$). These exist only off-axis ($B_r = 0$ exactly on the beam axis by symmetry, Eq. (2)), and they are what Eq. (5) actually integrates against as the time-dependent perturbation once $r \neq 0$. Two distinct ladder operators contribute: the $g_J \mu_B B_r$ terms (from J_x) connect states that differ by $\Delta m_J = \pm 1$ at fixed m_I – i.e. $\alpha_1 \leftrightarrow \beta_4$ and $\alpha_2 \leftrightarrow \beta_3$ – while the much smaller $g_I \mu_N B_r$ terms (from I_x , suppressed by $\mu_N/\mu_B \sim 1/1836$) connect states differing by $\Delta m_I = \pm 1$ at fixed m_J , i.e. $\alpha_1 \leftrightarrow \alpha_2$ and $\beta_3 \leftrightarrow \beta_4$. Both channels are pure *loss*: any population an atom sheds into them, off-axis, is population that fails to land cleanly on the intended final substate, exactly the “undesirable Larmor precession... and subsequent polarization losses” described for the off-axis case in [5]. The zero-crossing condition, Eq. (3), exists specifically to keep these B_r -driven entries too short-lived (traversed too suddenly relative to the Larmor precession they induce) to build up significant leakage.
- **The $H_{13} = H_{31} = 0$ entry (identically zero, on- or off-axis).** No linear combination of J_z, J_x, I_z, I_x (nor the zero-field flip-flop term) connects $|\alpha_1\rangle = |++\rangle$ directly to $|\beta_3\rangle = |--\rangle$, since doing so would require simultaneously raising/lowering *both* m_J and m_I by one unit in a single step – a second-order process entirely absent from Eq. (7). α_1 and β_3 **never couple to each other directly, at any radius, at any field profile.** What the literature calls the “ $\alpha_1 \rightarrow \beta_3$ transition” is therefore not a matrix element at all: it is the statement that $|\alpha_1\rangle$ is an *exact* eigenvector of Eq. (7) for any $B_z(t)$ on-axis (first row/column of the matrix reduces to a single diagonal entry when $B_r = 0$), so the amplitude simply sits still while the diagonal energy H_{11} sweeps past H_{33} during the reversal

– and is subsequently *read out* as “ β_3 ” because the downstream spinfilter’s holding field has itself reversed, not because any amplitude moved (Sec. 2.5).

A note on basis convention. Eq. (7) is written in the *uncoupled* product basis $\{|++\rangle, |+-\rangle, |--\rangle, |-+\rangle\}$ with the labels $\alpha_1, \alpha_2, \beta_3, \beta_4$ applied loosely to its members (as the text above already notes, $|2\rangle, |4\rangle$ are only the *high-field asymptotes* of α_2, β_4 , not the field-independent $B=0$ eigenstates of Table 6.1 in [1]). The implemented code (Sec. 2.5) instead diagonalizes H_0 *first* and works in its exact eigenbasis, so that α_2, β_4 are the fixed, field-independent combinations $(|+-\rangle \pm |-+\rangle)/\sqrt{2}$ at every field, not just asymptotically. The two matrices are unitarily equivalent – rotating Eq. (7) by the fixed 2×2 rotation in the $\{2, 4\}$ subspace reproduces the code’s matrix exactly, verified numerically to machine precision – and α_1, β_3 are identical, unambiguous objects in both conventions (that is exactly why they are called “stretched” states). This is a bookkeeping difference, not a physics disagreement; it is noted here only because the two matrices look superficially different (the constant $A/2$ entry above versus a purely B_z -dependent diagonal splitting in the code’s convention) and a reader comparing them directly should not mistake that for an inconsistency.

Eq. (7) is written with A left symbolic precisely because it applies unchanged to both cases of Table 2.1: substituting $A = A_{1S}$ gives the matrix relevant to actual OPPIS operation, while $A = A_{2S}$ gives the one that should reproduce the Lamb-shift-polarimeter bench data of Sec. 2.5. The only numbers that change between the two cases are A itself (and, through it, B_c and the diagonal splittings); the qualitative structure – which entries are diagonal, which are the constant $A/2$ flip-flop term, which are B_r -proportional, and which are identically zero – is exactly the same for ground and metastable atoms alike.

In short: Eq. (7) contains exactly two physically distinct mechanisms and one purely parasitic one. The $A/2$ entry drives a genuine, on-axis, B_z -only adiabatic population transfer between α_2 and β_4 (“Branch 2”); the identically-zero H_{13} entry, combined with the diagonal-only on-axis Hamiltonian, is what makes α_1 (“Branch 1”) simply sit frozen while the reversal happens around it; and the four B_r -proportional entries, which exist only off-axis, are the sole source of polarization loss, to be suppressed by satisfying Eq. (3).

2.5 Fixed vs. rotating quantization axis, and the two branches of the mechanism

Eq. (7) is written in a basis whose quantization axis is fixed to the lab z -axis at all times; this is the natural choice for a code that just integrates Eq. (5) directly, and it is what our implementation uses throughout. An alternative, equivalent bookkeeping rotates the quantization axis to always point along the *local* total field direction (used, e.g., in Fig. 4 of Antishev & Belov [4], as opposed to their Fig. 3, which – like our code – does not perform this rotation).

A conservation law, not just a matrix element. On axis ($B_r = 0$), the total angular-momentum projection $m_F = m_J + m_I$ is an *exactly* conserved quantum number: B_z couples only to J_z and I_z individually, and the zero-field flip-flop term conserves $m_J + m_I$ by construction, so $[H, J_z + I_z] = 0$ whenever $B_r = 0$, at any B_z . Since α_1 ($m_F = +1$) and β_3 ($m_F = -1$) carry *different* values of this exactly conserved quantity, no on-axis process, however fast or slow, can connect them – $H_{13} = 0$ in Eq. (7) is not a coincidence of this particular Hamiltonian, it is required by symmetry. α_2 and β_4 , by contrast, are *degenerate* in m_F (both $m_F = 0$), which is exactly why they are free to mix via the constant flip-flop term and are the only pair that can undergo genuine on-axis dynamics.

Branch 1 (alpha1): frozen, by symmetry. α_1 therefore never moves, on-axis, for any field history – confirmed both by this conservation argument and directly in code (Sec. 2.5). Since α_1 already carries $m_I = +1/2$, this branch requires no dynamics to already be “correct” for the purpose of nuclear polarization.

Branch 2 ($\alpha_2 \rightarrow \beta_4$): a genuine, on-axis, adiabatic spin flip. The α_2 - β_4 pair has a minimum energy gap of exactly A at $B_z = 0$ (Sec. 2.4) that never closes, so for essentially any realistic beam velocity this pair is crossed adiabatically: the atom stays on the same energy branch continuously through the reversal. Because that branch’s asymptotic (Paschen–Back) identity swaps sign of m_I on either side of the crossing, an atom entering as $|m_J=+1/2, m_I=-1/2\rangle$ (the high-field limit of α_2) exits as $|m_J=-1/2, m_I=+1/2\rangle$ (the high-field limit of β_4) – a real, physical flip of the nuclear spin projection, by the same adiabatic-fast-passage mechanism used to invert spins in NMR/ESR by sweeping a field through resonance. This is confirmed directly by the arrow diagram in Fig. 1 of Kannis et al. (2022) [5]: the caption states explicitly that the arrows “indicate the initial $|m_J, m_I\rangle$ and the final $|m'_J, m'_I\rangle$ substates that the eigenenergies approach,” with the α_2 branch carrying an explicit m_I -flip arrow while the straight α_1 line carries none.

Convergence: how the two branches together give full polarization transfer. Upstream of the Sona unit, a spin-selection stage leaves only α_1, α_2 populated, 50/50, with zero net nuclear polarization ($m_I = +1/2$ and $m_I = -1/2$ equally populated) but full electron polarization (both have $m_J = +1/2$). Through the Sona region, α_1 ends at $m_I = +1/2$ by *not moving at all* (Branch 1); α_2 ends at $m_I = +1/2$ by *genuinely, adiabatically flipping* (Branch 2). Two different mechanisms, the same final nuclear projection on both branches – this convergence is what converts electron polarization into nuclear polarization, and it requires only B_z : neither branch needs B_r .

Where B_r fits: a loss channel, not a third branch. B_r is not part of either mechanism above. Its role is exactly what Kannis et al. (2022) describe: an off-axis atom never actually sees $B = 0$, only a field that changes direction by 180° over a finite time, inducing “undesirable Larmor precession... and subsequent polarization losses.” Right at $B_z = 0$, α_1, α_2 , and the state about to become β_4 are all exactly degenerate (all at $A/4$ in H_0); a nonzero B_r can then freely mix across this vanishing gap, diluting the clean Branch-1/Branch-2 outcome by inducing unwanted transitions among $\alpha_1, \alpha_2, \beta_3$ (Kannis et al. 2022’s own “alternative explanation” paragraph: a time-varying radial component “induces transitions from β_3 to α_2 and from α_2 to α_1 ”). Sona’s zero-crossing condition, Eq. (3), exists specifically to suppress this off-axis leakage – *not* to enable Branch 1 or Branch 2, both of which proceed automatically, on-axis, regardless of speed (Branch 1 by symmetry; Branch 2 because the A -gap never closes).

Reconciling with the “pure states are exchanged” language in the literature. Sona (1967), Kponou et al. [3], and Engels et al. [2] describe the classical mechanism as an *exchange* of the pure/stretched states, $|F=1, m_F=+1\rangle \rightarrow |F=1, m_F=-1\rangle$. This is consistent with, not contradictory to, the frozen- α_1 picture above, once one notes that m_F in that statement is naturally defined *relative to the local holding field* at each end of the device (the physically relevant convention for an apparatus like a spinfilter, which itself measures relative to its own, local field direction) – i.e. it is the rotating-frame bookkeeping mentioned at the top of this section. Since the entrance and exit holding fields point in *opposite* directions (that is what a field reversal means), the same absolute, fixed-frame amplitude that never moved ($\alpha_1, m_F = +1$ relative to the fixed lab axis throughout) is correctly reported as “ $m_F = -1$,” i.e. β_3 , relative to the now-reversed exit holding field – with, as derived above, no Hamiltonian coupling ever invoked to produce that relabelling. The two forms of words (“frozen in the fixed frame,” “exchanged in the rotating frame”) describe the identical physical outcome; our code, and the analysis in this note, use the fixed-frame convention throughout because it is what a direct numerical integration of Eq. (5) naturally produces, and because Branches 1 and 2 are both already visible in it without any further transformation.

Two complementary pieces of code now exist. `MagScan_Ana4.py` (full listing in the Appendix) covers the field-map ingestion half of the pipeline described in Sec. 1: it takes the raw transversal magnetic field scans of the OPPIS Sona region (LCCS NewScanApp bench data: field vs. position, for a grid of correction-coil current LCC and solenoid currents ICC1/ICC2) and turns them into the physical quantities that condition the Sona transition. A new, independent sona Python package (also listed in the Appendix) now implements the quantum-mechanical propagation of Eq. (5) itself – the piece explicitly marked “not

yet implemented” in the previous version of this note. The two are not yet wired together (the propagation code currently reads a generic $(z, B_z[, B_r])$ CSV rather than `MagScan_Ana4.py`’s own data structures directly; Sec. 3 item 5), but they use a common field-derivative convention (Eq. (2)) and connecting them is a small, well-defined remaining step rather than new development.

2.6 Data ingestion and field-profile extraction

The raw CSV scan (columns: position, LCC, ICC1, ICC2, field) is cleaned (non-numeric footer rows dropped, ICC2= 0 points removed as a known power-supply-settling artifact) and split into the requested ICC2 plane. For every (LCC, ICC1) combination this gives a clean $B_z(z)$ profile over the 40–140 cm range of the bench.

2.7 Zero-crossing position and gradient

For each profile, the code locates all sign changes of $B_z(z)$ by linear interpolation between bracketing points, giving the crossing position x_0 and the local gradient dB_z/dz by a forward finite difference (this is exactly the quantity that enters the zero-crossing condition of Eq. (3)). Profiles with more than one crossing (which do occur, since the OPPIS Sona region is more complex than the idealized two-coil geometry of Ch. 3) are all recorded, with the highest- z crossing (the one the beam meets first) taken as the “primary” one for the summary plots.

2.8 Adiabaticity check against Sona’s threshold

`compute_sona_parameters()` evaluates the Kponou/Sona upper bound on $|dB_z/dz|$ (right-hand side of Eq. (3), H case) for a given beam kinetic energy and radius, so that the measured gradient at each crossing can be checked against it – a first, purely semiclassical sanity check that a given operating point is in the non-adiabatic (“sudden”) regime that Sona’s original argument requires.

2.9 Effective Sona frequency and energy-scale estimate

`compute_sona_freq()` implements a simple kinematic picture inspired by [1, 2]: an atom moving at velocity v through the radial field $B_r(z) = -\frac{1}{2}(dB_z/dz)r$ effectively experiences an oscillating transverse field between the two inflection points of $B_z(z)$ (i.e. the two zeros of dB_z/dz flanking the reversal), separated by a spatial wavelength λ . This defines an effective drive frequency $f = v/\lambda$ and a series of candidate transition energies $\Delta E_n = n\hbar f$ ($n = 1, 2, \dots$), which are plotted alongside the known Breit–Rabi splittings at low field for comparison. **This is a heuristic, order-of-magnitude proxy for the driving frequency seen by the atom – it is not a substitute for solving Eq. (5)**, but it is useful as a quick, model-independent check of which harmonic could be resonant with a given hyperfine splitting for a given coil setting, before running the full coupled-state calculation.

2.10 Correlation with measured polarization

The script also loads the independently measured beam polarization vs. LCC scans and overlays/correlates them with the field-derived quantities above (zero-crossing position, gradient, effective frequency): a set of summary, scatter, and phase-space plots (Sec. “Correlation plots A/B/C” in the code) lets us see, empirically, whether the observed polarization oscillations track the field geometry – which is the experimental observation (thesis Ch. 5) that originally motivated the full quantum treatment of Ch. 6.

2.11 Interactive inspection tools

For day-to-day use on the bench, the script provides interactive (cursor/slider-driven) figures to inspect a single $B_z(z)$, $dB_z/dz(z)$ or $B_r(z, r)$ profile for any (LCC, ICC1) pair, and grid summaries of the zero-crossing position and gradient across the whole scan.

2.12 Quantum propagation implementation (sona package)

The sona package implements Eq. (4)–(5) directly, as four small, independently tested modules (automated test suite, all executed tests passing; full listings in the Appendix; the numerical cross-check against an independent Mathematica implementation is given in Sec. 2.13):

constants.py Physical constants and a single swappable parameter set for A (and the derived B_c) that switches the entire calculation between the metastable and ground-state cases of Table 2.1 – one enum argument, not a separate code path.

hamiltonian.py Builds H_0 and $V(B_z, B_r, \phi)$ in the exact $B=0$ eigenbasis of H_0 (see the basis-convention note, Sec. 2.4). Validated three independent ways: (i) every one of the 16 matrix entries reproduces Kannis (2023) Eq. 6.23 term-for-term; (ii) inserting them into Eq. (5) reproduces Eq. 6.24a–d; (iii) numerically diagonalizing $H_0 + V(B_z, B_r=0)$ reproduces the closed-form Breit–Rabi energies (thesis Eq. 3.18a–d) to $\text{rtol} = 10^{-10}$, for both atom states, across a field sweep spanning $\pm 3B_c$ – the strongest available check, since it compares against a completely independent, closed-form calculation.

propagator.py Integrates Eq. (5) for a single atom at fixed (r, v) – but in the *Schrödinger*, not interaction, picture: the code tracks $a_k(t) = c_k(t) e^{-iE_k t/\hbar}$ directly, so that the explicit phase factors $e^{i\omega_{kj}t}$ of Eq. (5) never need to be computed or tracked separately. Since $|a_k(t)|^2 = |c_k(t)|^2$ at every t (the extra factor has unit modulus), this changes nothing measurable and removes one entire layer of bookkeeping from the numerical implementation; the interaction-picture form, Eq. (5), is still the right one for the analytic/by-hand arguments elsewhere in this note (the harmonic/photon picture of Sec. 4.4). Written out, the four coupled equations the code integrates are

$$\begin{aligned} i\hbar \dot{a}_1 &= \left[\frac{A}{4} - \frac{B_z}{2}(\alpha + \beta) \right] a_1 - \frac{B_r}{2\sqrt{2}}(\alpha + \beta)e^{-i\phi}a_2 - \frac{B_r}{2\sqrt{2}}(\alpha - \beta)e^{-i\phi}a_4 \\ i\hbar \dot{a}_2 &= -\frac{B_r}{2\sqrt{2}}(\alpha + \beta)e^{i\phi}a_1 + \frac{A}{4}a_2 - \frac{B_r}{2\sqrt{2}}(\alpha + \beta)e^{-i\phi}a_3 + \frac{B_z}{2}(\alpha - \beta)a_4 \\ i\hbar \dot{a}_3 &= -\frac{B_r}{2\sqrt{2}}(\alpha + \beta)e^{i\phi}a_2 + \left[\frac{A}{4} + \frac{B_z}{2}(\alpha + \beta) \right] a_3 + \frac{B_r}{2\sqrt{2}}(\alpha - \beta)e^{i\phi}a_4 \\ i\hbar \dot{a}_4 &= -\frac{B_r}{2\sqrt{2}}(\alpha - \beta)e^{i\phi}a_1 + \frac{B_z}{2}(\alpha - \beta)a_2 + \frac{B_r}{2\sqrt{2}}(\alpha - \beta)e^{-i\phi}a_3 - \frac{3A}{4}a_4 \end{aligned} \quad (8)$$

i.e. simply Eq. (7) read off row by row – note $\dot{a}_1$ has no a_3 term (and vice versa), the algebraic statement of $H_{13} = 0$.

These are solved, not with a generic adaptive integrator, but via piecewise-constant unitary evolution: the flight is chopped into steps small enough that $H(t)$ is effectively constant within each one, for which the Schrödinger equation has an *exact* solution,

$$\mathbf{a}(t + \Delta t) = \exp\left(-\frac{i}{\hbar}H\left(t + \frac{\Delta t}{2}\right)\Delta t\right)\mathbf{a}(t). \quad (9)$$

Since H is Hermitian, this is computed in three steps rather than a generic matrix exponential: (i) diagonalize H at the step’s midpoint, giving its four instantaneous eigenvalues E_k and eigenvectors $|\psi_k\rangle$; (ii) apply the phase $e^{-iE_k\Delta t/\hbar}$ to each eigenvector’s component – exact, since evolution under a genuinely constant Hamiltonian is just each eigenstate accumulating phase at its own rate; (iii) rotate back into the $(\alpha_1, \alpha_2, \beta_3, \beta_4)$ basis for the next step. This guarantees $\mathbf{a}$ stays normalized to machine precision at every step, regardless of step size – not a property that must be checked after the fact.

The step size Δt is not fixed a priori: the field is scanned in advance, the largest energy gap $\max_k |E_k - E_j|$ found anywhere along the whole trajectory is taken as the (deliberately conservative) worst case, and $\Delta t = 2\pi/(30\omega_{\max})$ is set to resolve that fastest oscillation with 30 samples per period; this one value is then held fixed for every step of that trajectory. Convergence was checked explicitly by varying the oversampling factor from $5\times$ to $240\times$: even the coarsest setting

agreed with the finest to 1 part in 10^6 , and the default ($30\times$) agreed with the finest to 2.5×10^{-8} – solidly converged, with comfortable margin, rather than assumed.

Cross-checked against an independent adaptive ODE solver (`scipy.integrate.solve_ivp`). Reproduces, directly and exactly, both branches of Sec. 2.5: α_1 population stays at exactly 1.0 on-axis for any field history (Branch 1), and $\alpha_2 \rightarrow \beta_4$ tracks the expected adiabatic-to-diabatic trend as velocity is scanned (Branch 2).

beam.py Integrates single-particle results over a Gaussian radial beam profile, $w(r) \propto (r/\sigma^2)e^{-r^2/2\sigma^2}$ (Simpson’s rule on a truncated grid, normalized by the grid’s own weight integral rather than an assumed exact 1). Validated: the weight normalizes to 1 (checked against an independent `scipy.integrate.quad`, not just the code’s own Simpson integration); beam-averaged populations sum to 1; $\sigma \rightarrow 0$ recovers the on-axis result; larger σ gives monotonically more B_r -driven leakage (Sec. 2.5), matching the physical trend argued for before the code was written.

A synthetic two-opposed-coil field (on-axis Biot–Savart for two circular loops) stands in for a real OPERA2D or bench-measured field map until one is connected (Sec. 3 item 5); the field-provider interface itself is generic (reads any $(z, B_z[, B_r])$ CSV, deriving B_r from B_z via Eq. (2) if not supplied directly), so swapping in a real map is a data change, not a code change.

2.13 Independent cross-check against a Mathematica implementation

A separate implementation (`metastable_H_time.nb`) integrates the *interaction-picture* equations, Eq. (5), with Mathematica’s `NDSolve` for metastable H: 1 keV, $\lambda = 20$ cm, $B_z = -B_{\max} \sin(2\pi t/T)$ with $B_{\max} = 10$ mT, $B_r = -(r/2) dB_z/dz$ with $z = vt$, $r = 1$ cm, one full period, eigenvalues and eigenvectors of H_0 hard-coded. It was compared with the sona package as follows.

- **Hamiltonian.** A Hamiltonian built independently from the spin operators, $H = A \mathbf{I} \cdot \mathbf{J} - (g_J \mu_B \mathbf{J} + g_I \mu_N \mathbf{I}) \cdot \mathbf{B}$, rotated into the $(\alpha_1, \alpha_2, \beta_3, \beta_4)$ basis, agrees with $H_0 + V$ of `hamiltonian.py` to $\sim 10^{-16} hA$ for several (B_z, B_r, ϕ) : same operator, same basis.
- **Dynamics.** Both `propagate()` (matrix exponential) and `propagate_solve_ivp_reference()` reproduce an independent DOP853 replica of the notebook to $\sim 10^{-5}$ in the final populations (Table 2.13); the norm is conserved to 10^{-9} . As expected, β_4 survival decreases with radius: 0.9999 ($r=0$), 0.9970 (2.5 mm), 0.9938 (5 mm), 0.9931 (1 cm), 0.9689 (2 cm).

Initial state	$ c_{\alpha_1} ^2$	$ c_{\alpha_2} ^2$	$ c_{\beta_3} ^2$	$ c_{\beta_4} ^2$
β_4	0.0006	0.0018	0.0045	0.9931
α_1	0.1360	0.7105	0.1490	0.0045
α_2	0.2517	0.0360	0.7105	0.0018
β_3	0.6117	0.2517	0.1360	0.0006

Remaining differences are conventions, not physics: the value of A (see the note in Sec. 2.1); the particle mass used for v (the notebook uses the H-atom mass, the package `M_PROTON`, so its v is 0.027% higher – the atomic mass is the physical choice); Schrödinger vs. interaction picture ($|a_k|^2 = |c_k|^2$); and hard-coded vs. computed eigensystem. The table is suitable as a regression test for the package.

Pipeline stage	Status
Field map ingestion & cleaning	✓ implemented (measured OPPIS bench scans, <code>MagScan_Ana4.py</code>)
Zero-crossing / gradient extraction	✓ implemented
Sona adiabaticity threshold (Eq. 3)	✓ implemented (semiclassical)
Effective frequency / energy-scale proxy	✓ implemented (kinematic estimate)
Measured-polarization overlay & correlation	✓ implemented
Breit–Rabi Hamiltonian $H_0 + V$ (build & diagonalize)	✓ implemented & validated (<code>hamiltonian.py</code>)
Generic field provider (CSV, any geometry)	✓ implemented (<code>fields/csv_field.py</code>)
Coupled FODE solver for $c_k(r, t)$ (Eq. 5)	✓ implemented & validated (<code>propagator.py</code>)
Integration over beam radial profile	✓ implemented & validated (<code>beam.py</code>)
Cross-check vs. independent Mathematica code	✓ agrees to $\sim 10^{-5}$ (Sec. 2.13)
Current sweep / simulated polarization vs. current	✗ not yet implemented – next step (Sec. 3)
Beam energy-spread averaging	✗ not yet implemented
Real (OPERA2D / bench-measured) field map as propagation input	✗ not yet connected
Deuterium ($I = 1$, 6-state)	✗ not yet implemented

3 Plan for further development

The static Hamiltonian, field interpolation, coupled-FODE propagation, and beam-radial-averaging steps of the original plan (items 1–3 and part of 5 below, in their original numbering) are now implemented and validated (Sec. 2.12). The remaining steps, updated in light of the two-branch mechanism of Sec. 2.5:

- (1) **Three-way diagnostic, not a single transfer number.** Before trusting any current scan, rework how populations are reported to separate the three physically distinct quantities identified in Sec. 2.5: α_1 fidelity (Branch 1, should stay ≈ 1 on-axis – already confirmed), $\alpha_2 \rightarrow \beta_4$ adiabatic fidelity (Branch 2, should track the Breit–Rabi curve and be essentially velocity-independent given the A -sized gap – already confirmed), and B_r -driven leakage among $\alpha_1, \alpha_2, \beta_3$ (the loss channel, expected to grow with beam radius and with how “sudden” the crossing is relative to the local Larmor precession, $\omega_L = \frac{e}{2m_e} B_r$). Lumping these into one “population that left α_1 ” number, as an earlier draft of this analysis did, obscures which physics is responsible for any given feature of a current-scan curve.
- (2) **Current scan and validation against the thesis.** Sweep the Sona coil current (equivalently, LC-C/ICC1 in the bench field maps), rescaling the field amplitude, and evaluate the beam-averaged propagation (item 1’s three quantities) at each setting. Validate against (i) the published simulated curves of [1] for a simplified field, and (ii) our own OPPIS polarization measurements already loaded by `MagScan_Ana4.py`, closing the loop between the field-map analysis of Sec. 2.5 and the

full quantum dynamics. This is the single remaining validation checkpoint before the code can be trusted for predictive OPPIS design work.

- (3) **Beam energy spread.** Every trajectory currently uses a single, exact beam kinetic energy. A real beam has a spread; since Branch 2's adiabaticity depends on velocity (Sec. 2.5), this needs to be added as a further weighted average, structurally identical to the existing radial (beam.py) average, before quantitative comparison to data.
- (4) **Connect to real field data.** Feed MagScan_Ana4.py's processed bench scans (or, longer-term, an OPERA2D finite-element export) into the propagation code's generic field-provider interface, replacing the synthetic two-coil test field used for validation so far – first for the simplified two-solenoid bench geometry, then for the full OPPIS geometry.
- (5) **Deuterium extension.** The current implementation supports hydrogen ($I = 1/2$, 4 states) only; deuterium ($I = 1$, 6 states, thesis Eq. 6.25) is a structurally similar, currently unimplemented extension, needed for the metastable-D bench measurements referenced in Sec. 2.5.
- (6) **Consolidate constants and documentation.** Adopt one value of A_{2S} and use the H-atom mass for v ; add the Mathematica notebook and the regression test of Table 2.13 to the repository; bring the repository README (which still quotes $B_c = 63$ mT for the metastable state instead of 6.34 mT) and the tracked code (batched Hamiltonian, beam.py) in line with this note.
- (7) **Packaging for the PSTP contribution.** Once validated, wrap the above into a small set of reusable functions with the interactive/summary plotting style already used in MagScan_Ana4.py, to produce the figures for the oral contribution at PSTP (Mito, Japan): measured field maps, simulated vs. measured polarization oscillations, and the physical interpretation in terms of the two-branch mechanism and its off-axis degradation.

Appendix: current code listing

Two complete, current pieces of code are reproduced below. First, the four modules of the sona quantum-propagation package (Sec. 2.12), covering the Hamiltonian, the propagator, and beam averaging. Second, MagScan_Ana4.py, covering the field-map processing, adiabaticity check, kinematic frequency estimate, and polarization-correlation stages. As noted in Sec. 3, the two are not yet wired together; connecting them is the main remaining software-integration step.

sona package: quantum propagation

```

1  """
2  constants.py
3  =====
4  Universal physical constants, plus the atom-specific hyperfine parameters
5  that the rest of the simulation depends on.
6
7  Why this file exists on its own
8  =====
9  The propagator, the Hamiltonian, the field providers -- none of them care
10 *which* hydrogen level they're simulating. The physics only differs through
11 one number, the hyperfine constant A (and, derived from it, the critical
12 field B_c = A / (|g_J| mu_B + g_I mu_N)):
13
14     metastable 2S1/2 (FZJ, Kannis/Engels validation)    A ~ 177.6 MHz
15     ground-state 1S1/2 (BNL OPPIS)                     A ~ 1420.4 MHz
16
17 Everything downstream should ask this module for a HyperfineParams object
18 rather than hard-coding A anywhere. Swapping metastable <=> ground state
19 is then a one-line change: pass a different AtomState to
20 get_hyperfine_params().
21

```

```

22 References
23 -----
24 Kannis (2023) thesis, Table 5.2 (A values), Sec. 3.1.1 (B_c definition).
25 Hydrogen ground-state hyperfine (21 cm line): A/h = 1420.405751 MHz
26 (CODATA / standard reference value).
27 """
28
29 from dataclasses import dataclass
30 from enum import Enum
31 import math
32
33 # -----
34 # Universal physical constants (SI units)
35 # -----
36 H_PLANCK = 6.62607015e-34      # J s      Planck constant
37 HBAR      = H_PLANCK / (2.0 * math.pi) # J s
38 MU_B      = 9.2740100783e-24    # J/T     Bohr magneton
39 MU_N      = 5.0507837461e-27    # J/T     nuclear magneton
40 E_CHARGE  = 1.602176634e-19    # C
41 M_ELECTRON = 9.1093837015e-31  # kg
42 M_PROTON   = 1.67262192369e-27 # kg
43
44 # g-factors (dimensionless)
45 G_J_S = -2.00231930436 # electron Lande g-factor for an S state (J=1/2, L=0)
46 G_I_P = 5.5856946893   # proton g-factor
47 G_I_D = 0.8574382338   # deuteron g-factor
48
49
50 # -----
51 # Atom / level selector
52 # -----
53 class AtomState(Enum):
54     """Which hydrogen level we're simulating."""
55     METASTABLE_2S = "metastable_2S1/2" # FZJ Lamb-shift setup (Kannis/Engels)
56     GROUND_1S     = "ground_1S1/2"     # BNL RHIC OPPIS
57
58
59 @dataclass(frozen=True)
60 class HyperfineParams:
61     """
62     Everything the Hamiltonian needs that depends on WHICH level is being
63     simulated. Immutable on purpose -- build a new one rather than mutating.
64     """
65     label: str
66     A_Hz: float # hyperfine constant A / h [Hz]
67     gJ: float # electron g-factor
68     gI: float # nuclear g-factor (proton or deuteron)
69     I_spin: float # nuclear spin quantum number (1/2 for H, 1 for D)
70
71     @property
72     def A_J(self) -> float:
73         """Hyperfine constant A in Joules (energy units, not A/h)."""
74         return self.A_Hz * H_PLANCK
75
76     @property
77     def B_critical_T(self) -> float:
78         """
79         Critical field B_c [Tesla] -- the field scale separating the
80         low-field (hyperfine-dominated) region from the high-field
81         (Zeeman-dominated, Paschen-Back) region.
82
83         
$$B_c = A / (|g_J| \mu_B + g_I \mu_N)$$

84
85         Kannis quotes B_c = 6.34 mT for metastable H; this formula
86         reproduces that value from A_Hz = 177.556 MHz.
87         """
88         denom = abs(self.gJ) * MU_B + self.gI * MU_N
89         return self.A_J / denom
90
91
92 # -----
93 # The swappable presets
94 # -----

```



```

95 HYPERFINE_PRESETS = {
96     AtomState.METASTABLE_2S: HyperfineParams(
97         label="metastable 2S1/2 (H)",
98         A_Hz=177.556e6,          # Kannis (2023), Table 5.2
99         gJ=G_J_S,
100        gI=G_I_P,
101        I_spin=0.5,
102    ),
103    AtomState.GROUND_1S: HyperfineParams(
104        label="ground state 1S1/2 (H)",
105        A_Hz=1420.405751e6,      # hydrogen 21 cm hyperfine transition
106        gJ=G_J_S,
107        gI=G_I_P,
108        I_spin=0.5,
109    ),
110 }
111
112
113 def get_hyperfine_params(state: AtomState = AtomState.METASTABLE_2S) -> HyperfineParams:
114     """
115     Single entry point for atom-dependent parameters.
116
117     Example
118     -----
119     >>> from sona.constants import AtomState, get_hyperfine_params
120     >>> p = get_hyperfine_params(AtomState.GROUND_1S)
121     >>> p.A_Hz, p.B_critical_T
122     """
123     return HYPERFINE_PRESETS[state]
124
125
126 if __name__ == "__main__":
127     # Quick sanity print -- run `python -m sona.constants` from src/.
128     for state in AtomState:
129         p = get_hyperfine_params(state)
130         print(f"{p.label:28s}  A = {p.A_Hz/1e6:9.3f} MHz      "
131               f"B_c = {p.B_critical_T*1e3:7.3f} mT")

```

Listing 1: constants.py — atomic parameters, with the swappable metastable/ground-state parameter set.

```

1  """
2  hamiltonian.py
3  =====
4  Builds the atomic Hamiltonian  $H(t) = H_0 + V(r,t)$  for the Sona-transition
5  propagator, in the FIXED  $B=0$  eigenbasis of the field-free hyperfine
6  Hamiltonian  $H_0 = A * I.J$ .
7
8  Basis (hydrogen,  $J = I = 1/2$ )
9  -----
10 The four basis states are the  $B=0$  limits of the field-dependent hyperfine
11 eigenstates  $\alpha_1, \alpha_2, \beta_3, \beta_4$  (Kannis Table 3.1 / Table 6.1):
12
13      $|1\rangle = |m_J=+1/2, m_I=+1/2\rangle$  (alpha1 @  $B=0$ )
14      $|2\rangle = (|m_J=-1/2, m_I=+1/2\rangle + |m_J=+1/2, m_I=-1/2\rangle) / \sqrt{2}$  (alpha2 @  $B=0$ )
15      $|3\rangle = |m_J=-1/2, m_I=-1/2\rangle$  (beta3 @  $B=0$ )
16      $|4\rangle = (|m_J=-1/2, m_I=+1/2\rangle - |m_J=+1/2, m_I=-1/2\rangle) / \sqrt{2}$  (beta4 @  $B=0$ )
17
18 States 1,2,3 are the  $F=1$  triplet ( $E=+A/4$ ); state 4 is the  $F=0$  singlet
19 ( $E=-3A/4$ ) -- the familiar hydrogen 21 cm hyperfine structure. This basis
20 is FIXED -- it does not change as the field changes. Everything the field
21 does is packed into  $V(r,t)$ .
22
23  $H_0$ 
24 ---
25 Diagonal in this basis:  $H_0 = \text{diag}(A/4, A/4, A/4, -3A/4)$ . Field-independent.
26
27  $V(r,t)$ 
28 -----
29 The Zeeman interaction  $-(g_J \mu_B J + g_I \mu_N I) \cdot B$ , with
30  $B = B_z(t) \hat{z} + B_r(r,t) \hat{r}(\phi)$ , expressed as matrix elements in the
31 basis above. Derived here directly from the  $J_+/J_-/I_+/I_-$  raising and
32 lowering algebra on the uncoupled  $|m_J, m_I\rangle$  states (spin-1/2 ladder

```

```

33 operators have no sqrt prefactor, which keeps this clean), then rotated
34 into the fixed basis.
35
36 This was cross-checked two independent ways against Kannis (2023):
37 1. Matrix form -- every one of the 16 entries reproduces Eq. 6.23.
38 2. ODE form -- inserting these matrix elements into Eq. 6.16 and
39    adding the phase factors  $e^{i\omega_{kj}t}$  reproduces Eq. 6.24a-d
40    term for term.
41 3. (Implemented as an automatic test, not just a claim.) Diagonalizing
42     $H_0 + V(Bz, Br=0)$  at any field must reproduce the closed-form
43    Breit-Rabi energies  $E_{1..4}(B)$ , Eq. 3.18a-d -- see
44    breit_rabi_energies_hydrogen() below and
45    tests/test_hamiltonian.py.
46
47 Populations are independent of the azimuthal angle  $\phi$  (Kannis Sec. 6.1);
48 it appears only as an overall phase. Callers may safely pass  $\phi=0$ .
49
50 Deuterium ( $I=1$ , 6-state, Kannis Eq. 6.25 / Sec. 6.1.2) is NOT yet
51 implemented -- functions below raise NotImplementedError for  $I \neq 1/2$ 
52 rather than silently returning wrong-sized matrices.
53 """
54
55 import numpy as np
56
57 from sona.constants import HyperfineParams, MU_B, MU_N
58
59 SQR2 = np.sqrt(2.0)
60
61 # Basis label reference, in the fixed  $B=0$  eigenbasis (hydrogen):
62 BASIS_LABELS_H = ("alpha1", "alpha2", "beta3", "beta4")
63
64 def _check_is_hydrogen(params: HyperfineParams) -> None:
65     if params.I_spin != 0.5:
66         raise NotImplementedError(
67             f"I_spin={params.I_spin} (deuterium-like) is not implemented. "
68             "This module currently supports hydrogen ( $I=1/2$ ) only -- see "
69             "Kannis Eq. 6.25 for the deuterium  $H_0$  this would need."
70         )
71
72
73 def H0_hydrogen(params: HyperfineParams) -> np.ndarray:
74     """
75     Field-free hyperfine Hamiltonian, diagonal in the fixed basis
76     (alpha1, alpha2, beta3, beta4) @  $B=0$ .
77
78     Returns
79     -----
80     (4,) real ndarray of eigenenergies [J], order matching BASIS_LABELS_H.
81     """
82     _check_is_hydrogen(params)
83     A = params.A_J
84     return np.array([A / 4.0, A / 4.0, A / 4.0, -3.0 * A / 4.0])
85
86
87 def interaction_matrix_hydrogen(Bz: float, Br: float, phi: float,
88                                params: HyperfineParams) -> np.ndarray:
89     """
90      $\langle k|V(r,t)|j\rangle$  for hydrogen -- 4x4 Hermitian, complex128, in the same
91     fixed basis as H0_hydrogen.
92
93     Parameters
94     -----
95     Bz, Br : float    Tesla. Longitudinal / radial field components.
96     phi    : float    radians. Azimuthal angle; populations don't depend
97                  on it (pass 0.0 unless you have a specific reason).
98     params : HyperfineParams
99
100     Returns
101     -----
102     (4,4) complex128 ndarray.
103     """
104     _check_is_hydrogen(params)

```

```

106 alpha = params.gJ * MU_B # electron magnetic-moment coefficient
107 beta = params.gI * MU_N # nuclear magnetic-moment coefficient
108 ephi_m = np.exp(-1j * phi)
109 ephi_p = np.exp(1j * phi)
110
111 V = np.zeros((4, 4), dtype=complex)
112
113 # -- diagonal (Bz only) --
114 V[0, 0] = -Bz / 2.0 * (alpha + beta)
115 V[1, 1] = 0.0
116 V[2, 2] = Bz / 2.0 * (alpha + beta)
117 V[3, 3] = 0.0
118
119 # -- (1,2) / (2,1): couples alpha1 <-> alpha2 via Br, (alpha+beta) --
120 V[0, 1] = -(Br / (2.0 * SQRT2)) * (alpha + beta) * ephi_m
121 V[1, 0] = np.conj(V[0, 1])
122
123 # -- (1,3) / (3,1): no direct coupling --
124 V[0, 2] = 0.0
125 V[2, 0] = 0.0
126
127 # -- (1,4) / (4,1): couples alpha1 <-> beta4 via Br, (alpha-beta) --
128 V[0, 3] = -(Br / (2.0 * SQRT2)) * (alpha - beta) * ephi_m
129 V[3, 0] = np.conj(V[0, 3])
130
131 # -- (2,3) / (3,2): couples alpha2 <-> beta3 via Br, (alpha+beta) --
132 V[1, 2] = -(Br / (2.0 * SQRT2)) * (alpha + beta) * ephi_m
133 V[2, 1] = np.conj(V[1, 2])
134
135 # -- (2,4) / (4,2): couples alpha2 <-> beta4 via Bz, (alpha-beta) --
136 V[1, 3] = Bz / 2.0 * (alpha - beta)
137 V[3, 1] = V[1, 3] # real -> Hermitian conjugate is itself
138
139 # -- (3,4) / (4,3): couples beta3 <-> beta4 via Br, (alpha-beta) --
140 V[2, 3] = (Br / (2.0 * SQRT2)) * (alpha - beta) * ephi_p
141 V[3, 2] = np.conj(V[2, 3])
142
143 return V
144
145
146 def build_H_hydrogen(Bz: float, Br: float, phi: float,
147                      params: HyperfineParams) -> np.ndarray:
148     """
149     Full Schrödinger-picture Hamiltonian  $H(t) = H_0 + V(r,t)$ , 4x4 complex128,
150     ready to hand to the propagator (matrix exponential or solve_ivp).
151     """
152     H0 = np.diag(H0_hydrogen(params)).astype(complex)
153     V = interaction_matrix_hydrogen(Bz, Br, phi, params)
154     return H0 + V
155
156
157 def breit_rabi_energies_hydrogen(B: float, params: HyperfineParams) -> np.ndarray:
158     """
159     Closed-form Breit-Rabi energies  $E1..E4(B)$  for hydrogen, on-axis field
160     only (Kannis Eq. 3.18a-d). Used as an independent analytic oracle to
161     validate build_H_hydrogen(Bz=B, Br=0, ...) -- NOT used by the
162     propagator itself.
163
164     Returns
165     -----
166     (4,) real ndarray: (E1, E2, E3, E4) matching (alpha1, alpha2, beta3,
167     beta4) at high field, in Joules.
168     """
169     _check_is_hydrogen(params)
170     A = params.A_J
171     alpha = params.gJ * MU_B
172     beta = params.gI * MU_N
173
174     E1 = A / 4.0 - 0.5 * (alpha + beta) * B
175     E3 = A / 4.0 + 0.5 * (alpha + beta) * B
176     disc = np.sqrt(A ** 2 + (alpha - beta) ** 2 * B ** 2)
177     E2 = -A / 4.0 + 0.5 * disc
178

```

```

179 E4 = -A / 4.0 - 0.5 * disc
180 return np.array([E1, E2, E3, E4])

```

Listing 2: hamiltonian.py — H_0 and the Zeeman interaction matrix V , in the exact $B=0$ eigenbasis.

```

1  """
2  propagator.py
3  =====
4  Propagates the hydrogen hyperfine-state amplitude vector a(t) through a
5  field trajectory Bz(z), Br(r,z), converting spatial position to time via
6  z = v*t for a particle moving at constant velocity v.
7
8  Method: piecewise-constant unitary propagation (short-time matrix
9  exponential). Over each small step dt, H(t) is evaluated at the step
10 midpoint and treated as constant; the exact propagator for a constant
11 Hermitian H is
12
13     a(t+dt) = exp(-i H dt / hbar) a(t)
14
15 computed via eigendecomposition of H (H is Hermitian, so this is exact
16 and numerically cheap for a 4x4 matrix). Because H is Hermitian at every
17 step, this propagator is unitary to machine precision REGARDLESS of step
18 size -- norm conservation is not something to check after the fact and
19 hope holds, it's guaranteed by construction. See
20 propagate_and_check_norm() and tests/test_propagator.py.
21
22 An independent cross-check against a generic adaptive ODE solver
23 (scipy.integrate.solve_ivp) is provided in
24 propagate_solve_ivp_reference() -- if the two methods disagree, that's
25 the thing to investigate before trusting either.
26
27 On-axis vs off-axis
28 -----
29 This module is written generally (radius r is a parameter of
30 propagate()), but for now we are only validating the r=0 (on-axis, Br=0)
31 case. At Br=0:
32 - alpha1 (index 0) and beta3 (index 2) are EXACT eigenstates at every
33 instant, for any Bz(t) -- V[0,2]=V[2,0]=0 identically and neither
34 couples to anything else without Br. A particle prepared in either
35 state shows zero population transfer, always. This is a strong,
36 exact correctness check on the Hamiltonian (see
37 test_alpha1_beta3_are_frozen_on_axis).
38 - alpha2 (index 1) and beta4 (index 3) ARE coupled via Bz alone
39 (V[1,3] != 0 with no Br needed) -- this is the real on-axis
40 physics: the avoided crossing visible as the two curved branches
41 in the Breit-Rabi diagram. Slow field reversal drives adiabatic
42 following (population tends to swap diabatic identity across the
43 crossing); fast reversal tends to preserve the initial diabatic
44 state. See test_alpha2_beta4_adiabatic_vs_diabatic_trend.
45 """
46
47 import numpy as np
48 from scipy.integrate import solve_ivp
49
50 from sona.constants import HyperfineParams, HBAR, E_CHARGE, M_PROTON
51 from sona.hamiltonian import build_H_hydrogen
52
53
54 def kinetic_energy_to_velocity(KE_eV: float, mass_kg: float = M_PROTON) -> float:
55     """
56     Non-relativistic KE -> speed [m/s]. Valid here: even at a few keV,
57     v/c ~ 1e-3 for a proton, so relativistic corrections are irrelevant
58     (gamma - 1 ~ 3e-6 at 3 keV).
59
60     Example: kinetic_energy_to_velocity(3000.0) -> proton speed at 3 keV.
61     """
62     KE_J = KE_eV * E_CHARGE
63     return np.sqrt(2.0 * KE_J / mass_kg)
64
65
66 def _unitary_step(H: np.ndarray, dt: float) -> np.ndarray:
67     """exp(-i H dt / hbar) via eigendecomposition. H must be Hermitian."""
68     eigvals, eigvecs = np.linalg.eigh(H)

```

```

69     phase = np.exp(-1j * eigvals * dt / HBAR)
70     return (eigvecs * phase) @ eigvecs.conj().T
71
72
73 def estimate_dt(field, v_mps: float, params: HyperfineParams, r: float = 0.0,
74                 phi: float = 0.0, oversample: int = 30, n_sample: int = 400) -> float:
75     """
76     Pick a fixed step size dt that resolves the fastest energy splitting
77     the trajectory will encounter, oversampled by `oversample`.
78
79     Scans the field's z-range, builds H at each sample point, and uses
80     the largest |Ek - Ej| found anywhere along the trajectory (a
81     conservative, not adaptive, choice -- simple and safe for a first
82     implementation).
83     """
84     z_min, z_max = field.z_range()
85     z_samples = np.linspace(z_min, z_max, n_sample)
86     max_gap = 0.0
87     for z in z_samples:
88         Bz = float(field.Bz(np.array([z]))[0])
89         Br = float(field.Br(r, np.array([z]))[0]) if r > 0 else 0.0
90         H = build_H_hydrogen(Bz, Br, phi, params)
91         eigvals = np.linalg.eigvalsh(H)
92         gap = eigvals[-1] - eigvals[0]
93         max_gap = max(max_gap, gap)
94
95     if max_gap <= 0.0:
96         # Degenerate/trivial field (e.g. Bz=0 everywhere) -- fall back to
97         # the hyperfine splitting A, which is always present.
98         max_gap = params.A_J
99
100     omega_max = max_gap / HBAR
101     return (2.0 * np.pi) / (oversample * omega_max)
102
103
104 def propagate(v_mps: float, field, params: HyperfineParams,
105              initial_state: np.ndarray, r: float = 0.0, phi: float = 0.0,
106              dt: float = None, oversample: int = 30,
107              return_trajectory: bool = False):
108     """
109     Propagate a single particle at radius r, speed v_mps, through the
110     field's full z-range, using piecewise-constant unitary steps.
111
112     Parameters
113     -----
114     v_mps : float           particle speed [m/s]
115     field : FieldProvider   e.g. a CSVFieldProvider
116     params : HyperfineParams
117     initial_state : (4,) complex array, should be normalized
118     r : float               radius [m]; r=0 means on-axis, Br forced to 0
119     phi : float             azimuthal angle [rad]; populations don't depend on it
120     dt : float or None      fixed step [s]; if None, computed via estimate_dt
121     oversample : int        passed to estimate_dt if dt is None
122     return_trajectory : bool if True, also return arrays of t, z, |a_k|^2
123
124     Returns
125     -----
126     a_final : (4,) complex ndarray
127     trajectory : dict (only if return_trajectory=True), keys 't', 'z', 'pop'
128     """
129     initial_state = np.asarray(initial_state, dtype=complex)
130     if not np.isclose(np.sum(np.abs(initial_state) ** 2), 1.0, atol=1e-9):
131         raise ValueError("initial_state must be normalized (sum |a_k|^2 = 1).")
132
133     z_min, z_max = field.z_range()
134     if dt is None:
135         dt = estimate_dt(field, v_mps, params, r=r, phi=phi, oversample=oversample)
136
137     t_span = (z_max - z_min) / v_mps
138     n_steps = int(np.ceil(t_span / dt))
139     dt = t_span / n_steps # exact fit, avoids overshooting z_max
140
141     a = initial_state.copy()

```

```

142
143     if return_trajectory:
144         t_arr = np.empty(n_steps + 1)
145         z_arr = np.empty(n_steps + 1)
146         pop_arr = np.empty((n_steps + 1, len(initial_state)))
147         t_arr[0], z_arr[0] = 0.0, z_min
148         pop_arr[0] = np.abs(a) ** 2
149
150     for i in range(n_steps):
151         t_mid = (i + 0.5) * dt
152         z_mid = z_min + v_mps * t_mid
153         Bz = float(field.Bz(np.array([z_mid]))[0])
154         Br = float(field.Br(r, np.array([z_mid]))[0]) if r > 0 else 0.0
155
156         H_mid = build_H_hydrogen(Bz, Br, phi, params)
157         U = _unitary_step(H_mid, dt)
158         a = U @ a
159
160     if return_trajectory:
161         t_arr[i + 1] = (i + 1) * dt
162         z_arr[i + 1] = z_min + v_mps * t_arr[i + 1]
163         pop_arr[i + 1] = np.abs(a) ** 2
164
165     if return_trajectory:
166         return a, {"t": t_arr, "z": z_arr, "pop": pop_arr}
167     return a
168
169
170 def propagate_solve_ivp_reference(v_mps: float, field, params: HyperfineParams,
171                                   initial_state: np.ndarray, r: float = 0.0,
172                                   phi: float = 0.0, rtol: float = 1e-10,
173                                   atol: float = 1e-12) -> np.ndarray:
174     """
175     Independent cross-check: integrate the same  $i\hbar da/dt = H(t)a$ 
176     with a generic adaptive solver (solve_ivp, DOP853), rather than the
177     fixed-step matrix-exponential method. solve_ivp doesn't accept
178     complex state vectors directly, so the 4 complex amplitudes are
179     packed into 8 real numbers and unpacked afterward.
180     """
181     initial_state = np.asarray(initial_state, dtype=complex)
182     z_min, z_max = field.z_range()
183     t_span = (0.0, (z_max - z_min) / v_mps)
184
185     def rhs(t, y):
186         a = y[:4] + 1j * y[4:]
187         z = z_min + v_mps * t
188         Bz = float(field.Bz(np.array([z]))[0])
189         Br = float(field.Br(r, np.array([z]))[0]) if r > 0 else 0.0
190         H = build_H_hydrogen(Bz, Br, phi, params)
191         da_dt = (H @ a) / (1j * HBAR)
192         return np.concatenate([da_dt.real, da_dt.imag])
193
194     y0 = np.concatenate([initial_state.real, initial_state.imag])
195     sol = solve_ivp(rhs, t_span, y0, method="DOP853", rtol=rtol, atol=atol)
196     yf = sol.y[:, -1]
197     return yf[:4] + 1j * yf[4:]

```

Listing 3: propagator.py — the matrix-exponential unitary propagator.

```

1  """
2  beam.py
3  =====
4  Averages single-particle propagator results over a realistic beam's
5  radial (Gaussian) profile.
6
7  A real beam is not one particle at one radius -- it's a distribution,
8  and (as the exploratory off-axis scans showed) different radii pick up
9  different amounts of Br-driven population transfer. What a detector
10 measures is the population averaged over the whole beam, weighted by
11 how many particles are at each radius. This module does that averaging.
12
13 Radial weight
14 -----

```



```

15 For a 2D Gaussian beam profile with width sigma, the marginal density in
16 radius (after integrating out the azimuthal angle) is the Rayleigh-type
17 distribution
18
19     w(r) = (r / sigma^2) * exp(-r^2 / (2 sigma^2))
20
21 which integrates to 1 over r in [0, inf) -- see
22 test_gaussian_weight_normalizes_to_1. This is Kannis's convention (sigma
23 is the beam's characteristic radial width).
24
25 Method
26 -----
27 propagate() is called at a grid of radii spanning [0, r_max] (r_max
28 truncated at a few sigma, since the Gaussian weight decays fast beyond
29 that), and the resulting |a_k(r)|^2 are integrated against w(r) using
30 Simpson's rule. The result is normalized by the ACTUAL integral of w(r)
31 over the truncated range (not assumed to be exactly 1), so truncation
32 error shows up as a small, controlled correction rather than a silent
33 population leak -- see test_beam_average_populations_sum_to_one.
34
35 Performance note: estimate_dt scans the whole field and diagonalizes H
36 at ~400 points -- expensive if repeated per radius. dt is computed ONCE
37 (at r_max, a conservative/safe choice) and reused for every radius in
38 the grid, rather than re-estimated n_r times.
39 """
40
41 import numpy as np
42 from scipy.integrate import simpson
43
44 from sona.constants import HyperfineParams
45 from sona.propagator import propagate, estimate_dt
46
47
48 def gaussian_radial_weight(r: np.ndarray, sigma_m: float) -> np.ndarray:
49     """
50     w(r) = (r/sigma^2) * exp(-r^2 / (2 sigma^2)), the radial marginal of
51     a 2D Gaussian beam. Integrates to 1 over r in [0, inf).
52     """
53     r = np.asarray(r, dtype=float)
54     return (r / sigma_m ** 2) * np.exp(-r ** 2 / (2.0 * sigma_m ** 2))
55
56
57 def beam_average_populations(v_mps: float, field, params: HyperfineParams,
58                             initial_state: np.ndarray, sigma_m: float,
59                             r_max_sigma: float = 5.0, n_r: int = 81,
60                             phi: float = 0.0, return_details: bool = False):
61     """
62     Beam-averaged exit populations |c_k|^2, Gaussian-weighted over radius.
63
64     Parameters
65     -----
66     v_mps, field, params, initial_state, phi : as in propagator.propagate
67     sigma_m : float                beam radial width [m]
68     r_max_sigma : float            truncate the radial integral at r_max_sigma * sigma_m
69     n_r : int                     number of radii sampled (Simpson's rule)
70     return_details : bool         if True, also return the per-radius raw data
71
72     Returns
73     -----
74     avg_pop : (4,) ndarray        beam-averaged |c_k|^2, normalized to sum to 1
75     details : dict (only if return_details=True)
76         'r' : (n_r,) radii sampled [m]
77         'pop_vs_r' : (n_r, 4) raw single-particle populations at each r
78         'weight' : (n_r,) the (unnormalized) Gaussian weight at each r
79     """
80     r_max = r_max_sigma * sigma_m
81     r_grid = np.linspace(0.0, r_max, n_r)
82     weight = gaussian_radial_weight(r_grid, sigma_m)
83
84     # estimate_dt scans the whole field and diagonalizes H at ~400 points
85     # -- expensive. Do it ONCE (at r_max, the largest gap we'll see across
86     # the grid, so it's a safe/conservative choice) and reuse the same dt
87     # for every radius, instead of re-running the scan n_r times.

```

```

88     dt = estimate_dt(field, v_mps, params, r=r_max, phi=phi)
89
90     pop_vs_r = np.empty((n_r, len(initial_state)))
91     for i, r in enumerate(r_grid):
92         a_final = propagate(v_mps, field, params, initial_state, r=r, phi=phi, dt=dt)
93         pop_vs_r[i] = np.abs(a_final) ** 2
94
95     weight_integral = simpson(weight, x=r_grid)
96     avg_pop = np.array([
97         simpson(weight * pop_vs_r[:, k], x=r_grid) / weight_integral
98         for k in range(pop_vs_r.shape[1])
99     ])
100
101     if return_details:
102         return avg_pop, {"r": r_grid, "pop_vs_r": pop_vs_r, "weight": weight}
103     return avg_pop

```

Listing 4: beam.py — Gaussian radial beam averaging.

Field-map processing: MagScan_Ana4.py

```

1  """
2  =====
3  Transversal Magnetic Field Scan – LCC / ICC1 Analysis
4  Brookhaven National Laboratory – LCCS NewScanApp
5  =====
6
7  File layout
8  -----
9  Columns : Position (cm) | LCC (A) | ICC1 (A) | ICC2 (A) | Field (G)
10
11 Scan structure (current format – magsan_20260515_2143 and successors)
12 -----
13 LCC [A] : swept over a range (e.g. 70, 85, 100, 115, 130)
14 ICC1 [A] : a small set of values (typically –35)
15 ICC2 [A] : several values present in the file, but only ONE is used
16             for analysis (selected by ICC2_SELECT below).
17             ICC2 = 0 is always inhibited (unsettled power supply).
18 Position [cm] : 40 → 140 cm, uniform 1-cm steps
19
20 Analysis goal
21 -----
22 For every (LCC, ICC1) combination at the selected ICC2:
23 • find the zero-crossing of Bz(z) – linearly interpolated
24 • compute dBz/dz at that crossing – forward finite difference
25 Then display a summary grid:
26     rows    = ICC1 values
27     col 0    = zero-crossing position    x [cm] vs LCC
28     col 1    = field gradient dBz/dz    [G/cm] vs LCC
29
30 Module structure
31 -----
32 0. Physics helpers          – field_derivative, dBdz_at_crossing,
33                             compute_sona_parameters
34 1. load_data()              – load, clean, select ICC2 plane
35 2. scan_conditions()        – find x and dBdz for every (LCC, ICC1)
36 3. plot_profile()           – single-profile two-panel inspector
37 4. plot_summary_grid()      – main output: n_ICC1 × 2 grid of line plots
38 5. plot_profiles_grid()     – Fig 1a: Bz(z) panel grid, rows=ICC1,
39                             one line per LCC (colormapped)
40 6. plot_gradients_grid()    – Fig 1b: dBz/dz(z) same layout as Fig 1a
41
42 Author : <your name>
43 Date   : 2026-05
44 =====
45 """
46
47 # -----
48 # 0. Imports
49 # -----
50 import os

```

```

51 import numpy as np
52 import pandas as pd
53 import matplotlib.pyplot as plt
54 import matplotlib.ticker as ticker
55
56
57 # -----
58 # 1. Configuration – edit only this block to adapt the script
59 # -----
60
61 # ---- Data file -----
62 DATA_DIR = (r"C:/Users/acannavo/OneDrive – Brookhaven National Laboratory"
63             r"/LCCScan/NewScanApp/RawData/20260521")
64 DATA_FILE = "magspan_20260521_1451.csv"
65 DATA_PATH = os.path.join(DATA_DIR, DATA_FILE)
66
67 # ---- ICC2 selection -----
68 #
69 # Only one ICC2 value is used for the main analysis.
70 # ICC2 = 0 is always removed (power-supply settling artifact).
71 # Set ICC2_SELECT to the value you want to analyse.
72 # If the exact value is not found in the file, the script raises a clear error.
73 #
74 ICC2_SELECT = 2.5      # [A] – the ICC2 plane to analyse
75
76 # ---- Default parameters for the single-profile inspector -----
77 DEFAULT_LCC = 100.0    # [A]
78 DEFAULT_ICC1 = 0.492   # [A]
79
80 # ---- Column names -----
81 COL_POS = "Position (cm)"
82 COL_LCC = "LCC (A)"
83 COL_ICC1 = "ICC1 (A)"
84 COL_ICC2 = "ICC2 (A)"
85 COL_FIELD = "Field (G)"
86
87 # ---- Sona / beam parameters -----
88 BEAM_ENERGY_KEV = 2.5  # [keV]
89 BEAM_RADIUS_CM = 1.0   # [cm]
90
91 # ---- Resonance field -----
92 B_RESONANCE = 634.0    # [G] – spin-flipper Larmor resonance
93
94 # ---- Polarization data file -----
95 #
96 # Beam polarization scan: POL. AVG. vs LCC (A) for ICC1=0.492, ICC2=2.5.
97 # Each row contains 20 individual polarization measurements; the mean and
98 # standard error of the mean are computed on load.
99 #
100 POL_DATA_DIR = (r"C:/Users/acannavo/OneDrive – Brookhaven National Laboratory"
101                r"/LCCScan/NewScanApp/RawData/20260521")
102 POL_DATA_FILE = "scan_2025-11-15_lcc_200A_2cm_21cm_ave.csv"
103 POL_DATA_PATH = os.path.join(POL_DATA_DIR, POL_DATA_FILE)
104 POL_ICC1 = 0.492      # [A] ICC1 value for this polarization scan
105 POL_ICC2 = 2.5        # [A] ICC2 value for this polarization scan
106
107 # ---- Sona frequency / energy analysis -----
108 #
109 # Kannis et al. (2022) framework: atoms moving at  $v_{\text{atom}}$  experience  $B_r(z)$ 
110 # as a time-varying field of frequency  $f = v_{\text{atom}} / \lambda$ , where  $\lambda$  is the
111 # spatial distance between the two zeros of  $B_r$  flanking the reversal.
112 # This can drive hyperfine transitions with  $\Delta E = n \cdot h \cdot f$ .
113 #
114 SONA_BEAM_ENERGY_KEV = 2.002    # [keV] proton beam kinetic energy
115 SONA_N_MAX = 5                  # max integer multiple  $n$  to display
116 SONA_BR_SEARCH_ZMIN = 60.0      # [cm] lower bound of reversal search window
117 SONA_BR_SEARCH_ZMAX = 92.0      # [cm] upper bound of reversal search window
118 #
119 # Known Breit-Rabi transition energies at low field [neV] – add yours here.
120 # These are drawn as horizontal reference lines on the  $\Delta E$  vs LCC plot.
121 # Format: (energy_neV, label_string)
122 BREIT_RABI_REFS = [
123     (14.624, "ΔE Jülich (n=1)"), # from Kannis et al. for their geometry

```

```

124     (29.248, "ΔE Jülich (n=2)"),
125     (43.872, "ΔE Jülich (n=3)"),
126     (58.496, "ΔE Jülich (n=4)"),
127     (87.744, "ΔE Jülich (n=5)")
128 ]
129
130 # ---- Output control -----
131 SAVE_FIGURES = True      # write PNG files alongside the script
132
133 # ---- Plot style -----
134 FIGSIZE_PROFILE = (11, 7)
135 LINE_COLOR      = "steelblue"
136 LINE_WIDTH      = 1.8
137 ZERO_LINE_KW    = dict(color="grey", linewidth=0.8, linestyle="--")
138
139 # Crossing series style – shared by Fig 1a, Fig 1b, and Fig 3 (summary grid)
140 # Index 0 = 1st crossing (highest z), 1 = 2nd, 2 = 3rd, ...
141 SERIES_STYLE = [
142     dict(color="steelblue", marker="o", ms=6, lw=1.8, ls="-", label="1st crossing"),
143     dict(color="darkorange", marker="s", ms=6, lw=1.5, ls="--", label="2nd crossing"),
144     dict(color="seagreen", marker="D", ms=5, lw=1.5, ls=":", label="3rd crossing"),
145     dict(color="grey", marker="+", ms=7, lw=1.2, ls=":", label="4th+ crossing"),
146 ]
147
148
149 # =====
150 # PHYSICS HELPERS
151 # =====
152
153 def field_derivative(pos: np.ndarray, fld: np.ndarray):
154     """
155     Forward finite difference of the field profile.
156
157     Returns
158     -----
159     pos_mid : ndarray (N-1,) midpoint positions [cm]
160     dBdz    : ndarray (N-1,) (B[i+1]-B[i])/(z[i+1]-z[i]) [G/cm]
161     """
162     assert len(pos) == len(fld)
163     dz = np.diff(pos)
164     assert np.all(dz > 0), "pos must be strictly ascending"
165     return 0.5 * (pos[:-1] + pos[1:]), np.diff(fld) / dz
166
167
168 def dBdz_at_crossing(pos: np.ndarray, fld: np.ndarray, i: int) -> float:
169     """Forward-difference gradient at zero-crossing bracket index i."""
170     return float((fld[i + 1] - fld[i]) / (pos[i + 1] - pos[i]))
171
172
173 def compute_sona_parameters(energy_keV: float = None,
174                             radius_cm: float = None) -> dict:
175     """
176     Compute Sona adiabatic spin-flip threshold (Kponou et al., AIP 2008).
177
178     Returns dict with keys: energy_keV, radius_cm, v, tau,
179     omega_B_peak, dBdz_sona_Gcm.
180     """
181     if energy_keV is None: energy_keV = BEAM_ENERGY_KEV
182     if radius_cm is None: radius_cm = BEAM_RADIUS_CM
183
184     m_p = 1.6726219e-27; m_e = 9.10938e-31; e_C = 1.60218e-19
185     E_J = energy_keV * 1e3 * e_C
186     v = np.sqrt(2.0 * E_J / m_p)
187     r_m = radius_cm * 1e-2
188
189     dBdz_sona_Gcm = 8.0 * m_e * v / (e_C * r_m ** 2) * 100.0 # T/m → G/cm
190     tau = r_m / (2.0 * v)
191
192     params = dict(energy_keV=energy_keV, radius_cm=radius_cm,
193                  v=v, tau=tau, omega_B_peak=1.0/tau,
194                  dBdz_sona_Gcm=dBdz_sona_Gcm,
195                  _m_e=m_e, _e_C=e_C)
196     print(f"[sona] E={energy_keV} keV r={radius_cm} cm ")

```

```

197         f"v={v:.3e} m/s dBdz_threshold={dBdz_sona_Gcm:.3f} G/cm")
198     return params
199
200
201 # =====
202 # FUNCTION 1 – load_data
203 # =====
204
205 def load_data(path: str = DATA_PATH,
206              icc2_select: float = ICC2_SELECT) -> pd.DataFrame:
207     """
208     Load the raw CSV, clean it, and return only the selected ICC2 plane.
209
210     Steps
211     -----
212     1. Read CSV; coerce all columns to numeric (drops the '#Exit ...' footer).
213     2. Drop rows with any NaN in the key columns.
214     3. Remove ICC2 = 0 rows (power-supply settling artifact).
215     4. Keep only rows where ICC2 ≈ icc2_select.
216     5. Sort by (LCC, ICC1, Position) ascending.
217
218     Parameters
219     -----
220     path : str    Path to the CSV file.
221     icc2_select : float  ICC2 value to retain [A].
222
223     Returns
224     -----
225     pd.DataFrame  Cleaned, single-ICC2-plane data frame.
226     """
227     df = pd.read_csv(path)
228
229     for col in [COL_POS, COL_LCC, COL_ICC1, COL_ICC2, COL_FIELD]:
230         df[col] = pd.to_numeric(df[col], errors="coerce")
231
232     df = df.dropna(subset=[COL_POS, COL_LCC, COL_ICC1, COL_ICC2, COL_FIELD])
233
234     # Always remove ICC2 = 0 (unsettled power supply)
235     n_before = len(df)
236     df = df[np.abs(df[COL_ICC2]) > 1e-6].reset_index(drop=True)
237     print(f"[load_data]  Removed {n_before - len(df)} ICC2=0 rows.")
238
239     # Select the single ICC2 plane
240     available_icc2 = sorted(df[COL_ICC2].unique())
241     mask_icc2 = np.abs(df[COL_ICC2] - icc2_select) < 1e-6
242     if not mask_icc2.any():
243         raise ValueError(
244             f"ICC2_SELECT={icc2_select} not found in file.\n"
245             f"Available ICC2 values (after removing 0): {available_icc2}"
246         )
247     df = df[mask_icc2].reset_index(drop=True)
248
249     df = df.sort_values([COL_LCC, COL_ICC1, COL_POS]).reset_index(drop=True)
250
251     lcc_vals = sorted(df[COL_LCC].unique())
252     icc1_vals = sorted(df[COL_ICC1].unique())
253     pos_range = f"{df[COL_POS].min():.0f} → {df[COL_POS].max():.0f} cm"
254
255     print(f"[load_data]  Loaded {len(df)} rows | ICC2 = {icc2_select} A")
256     print(f"                    LCC : {lcc_vals}")
257     print(f"                    ICC1 : {icc1_vals}")
258     print(f"                    Pos  : {pos_range} ({df[COL_POS].nunique()} points)")
259
260     return df
261
262
263 # =====
264 # FUNCTION 2 – filter_profile
265 # =====
266
267 def filter_profile(df: pd.DataFrame,
268                  lcc: float,
269                  icc1: float,

```



```

416 result = pd.DataFrame(records)
417 print(f"\n[scan_conditions] {len(result)} combinations scanned.")
418 print(f"  Single crossing : {(result.n_crossings == 1).sum()}")
419 print(f"  Multi  crossing : {(result.n_crossings > 1).sum()}")
420 print(f"  No      crossing : {(result.n_crossings == 0).sum()}")
421 return result
422
423
424
425 # =====
426 # FUNCTION 5 – plot_profile (static single-profile inspector)
427 # =====
428
429 def plot_profile(df: pd.DataFrame,
430                 lcc: float = DEFAULT_LCC,
431                 icc1: float = DEFAULT_ICC1) -> None:
432     """
433     Static two-panel figure for one (LCC, ICC1) profile:
434     Top    : Bz(z) on a symlog y-scale with zero-crossing markers
435     Bottom : dBz/dz(z) step plot with crossing annotation
436
437     X-axis is inverted (140 → 40 cm) to match beam-line convention.
438
439     Usage
440     -----
441     >>> plot_profile(df, lcc=100, icc1=0.325)
442     """
443     profile = filter_profile(df, lcc=lcc, icc1=icc1)
444     crossings = find_zero_crossings(profile)
445     icc2_val = profile[COL_ICC2].iloc[0]
446
447     pos = profile[COL_POS].values
448     field = profile[COL_FIELD].values
449     mid_pos, dBdz_arr = field_derivative(pos, field)
450
451     fig, (ax_b, ax_d) = plt.subplots(
452         2, 1, figsize=FIGSIZE_PROFILE,
453         sharex=True,
454         layout="constrained",
455         gridspec_kw={"height_ratios": [2, 1], "hspace": 0.08}
456     )
457     title = (f"LCC = {lcc:.0f} A | ICC1 = {icc1:.3f} A "
458             f"| ICC2 = {icc2_val:.3g} A")
459     fig.suptitle(f"Longitudinal Field Profile & Gradient\n{title}", fontsize=13)
460
461     # ---- top: Bz(z) -----
462     ax_b.plot(pos, field, color=LINE_COLOR, linewidth=LINE_WIDTH, zorder=3)
463     ax_b.axhline(0, **ZERO_LINE_KW)
464     ax_b.axhline(B_RESONANCE, color="darkorange", lw=1.0, ls="--",
465                 alpha=0.8, label=f"±{B_RESONANCE:.0f} G")
466     ax_b.axhline(-B_RESONANCE, color="darkorange", lw=1.0, ls="--", alpha=0.8)
467
468     LINTHRESH = 50
469     ax_b.set_yscale("symlog", linthresh=LINTHRESH, linscale=0.5)
470     ax_b.yaxis.set_minor_locator(
471         ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH,
472                                     subs=np.arange(2, 10)))
473
474     if not crossings.empty:
475         ax_b.scatter(crossings["position_cm"], np.zeros(len(crossings)),
476                     color="crimson", marker="o", s=60, zorder=5,
477                     label="Zero crossing")
478         for _, row in crossings.iterrows():
479             ax_b.axvline(row["position_cm"], color="crimson",
480                         lw=0.8, ls="--", alpha=0.5)
481
482     ax_b.set_ylabel("Bz(z) [G] [symlog]", fontsize=11)
483     ax_b.tick_params(which="both", direction="in", top=True, right=True)
484     ax_b.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
485     ax_b.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
486     ax_b.legend(fontsize=9, loc="upper left")
487
488     # ---- bottom: dBz/dz(z) -----

```

```

489 ax_d.step(pos[:-1], dBdz_arr, where="post",
490           color="darkorange", linewidth=LINE_WIDTH, zorder=3)
491 ax_d.hlines(dBdz_arr[-1], pos[-2], pos[-1],
492            colors="darkorange", linewidth=LINE_WIDTH, zorder=3)
493 ax_d.axhline(0, **ZERO_LINE_KW)
494
495 if not crossings.empty:
496     for _, row in crossings.iterrows():
497         ax_d.scatter(row["position_cm"], row["dBdz_Gcm"],
498                     color="crimson", s=60, zorder=5)
499         ax_d.axvline(row["position_cm"], color="crimson",
500                     lw=0.8, ls="--", alpha=0.5)
501         ax_d.annotate(f" {row['dBdz_Gcm']:.2f} G/cm",
502                      xy=(row["position_cm"], row["dBdz_Gcm"]),
503                      fontsize=8, color="crimson", va="center")
504
505 ax_d.set_ylabel(r"$dB_z/dz$ (G/cm)", fontsize=11)
506 ax_d.set_xlabel("Position (cm)", fontsize=11)
507 ax_d.tick_params(which="both", direction="in", top=True, right=True)
508 ax_d.xaxis.set_minor_locator(ticker.AutoMinorLocator())
509 ax_d.yaxis.set_minor_locator(ticker.AutoMinorLocator())
510 ax_d.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
511
512 ax_d.set_xlim(pos.max(), pos.min()) # beam direction: 140 → 40 cm
513
514 if SAVE_FIGURES:
515     fname = f"profile_LCC{lcc:.0f}_ICCl{iccl:.3f}.png"
516     fig.savefig(fname, dpi=130, bbox_inches="tight")
517     print(f"[plot_profile] Saved {fname}")
518 plt.show()
519
520
521 # =====
522 # FUNCTION 5b – plot_profile_interactive
523 # =====
524
525 def plot_profile_interactive(df: pd.DataFrame,
526                             lcc: float = DEFAULT_LCC,
527                             iccl: float = DEFAULT_ICC1) -> None:
528     """
529     Interactive two-panel profile inspector with:
530
531     • Shared vertical cursor – move the mouse over either panel to read out
532       the interpolated Bz and dBz/dz values at that position. The cursor
533       line spans both panels simultaneously (sharex keeps them locked).
534       A text box in the top-right corner shows:
535         z = <position> cm
536         Bz = <field> G
537         dBz/dz = <gradient> G/cm
538
539     • LCC slider – discrete steps through all LCC values in the data.
540     • ICC1 slider – discrete steps through all ICC1 values in the data.
541       Changing either slider redraws both panels in-place (no figure
542       recreation) so the cursor and sliders remain live.
543
544     Requirements
545     -----
546     Requires an interactive matplotlib backend. In Spyder:
547       Tools → Preferences → IPython console → Graphics → Backend → Qt5
548     Or run before calling:
549       %matplotlib qt5
550
551     Usage
552     -----
553     >>> plot_profile_interactive(df)
554     >>> plot_profile_interactive(df, lcc=115, iccl=0.650)
555     """
556     from matplotlib.widgets import Slider
557     from matplotlib.gridspec import GridSpec
558
559     lcc_vals = sorted(df[COL_LCC].unique())
560     iccl_vals = sorted(df[COL_ICC1].unique())
561     iccl_val = df[COL_ICC2].iloc[0]

```

```

562
563 def _nearest_idx(val, arr):
564     a = np.array(arr)
565     return int(np.argmax(np.abs(a - val)))
566
567 # Clamp initial values to available grid points
568 lcc_idx = _nearest_idx(lcc, lcc_vals)
569 iccl_idx = _nearest_idx(iccl, iccl_vals)
570
571 # --- Figure layout via GridSpec ---
572 # Rows: 0=Bz panel (tall), 1=dBdz panel (shorter), 2=spacer, 3=sliders
573 fig = plt.figure(figsize=(11, 8))
574 gs = GridSpec(4, 1, figure=fig,
575             height_ratios=[2.8, 1.4, 0.15, 0.55],
576             hspace=0.08)
577
578 ax_b = fig.add_subplot(gs[0]) # Bz(z)
579 ax_d = fig.add_subplot(gs[1], sharex=ax_b) # dBz/dz, shared x
580 ax_sl_lcc = fig.add_subplot(gs[3]) # placeholder – split below
581
582 # Split the slider row into two equal halves manually
583 fig.delaxes(ax_sl_lcc)
584 ax_sl_lcc = fig.add_axes([0.12, 0.045, 0.33, 0.030])
585 ax_sl_iccl = fig.add_axes([0.57, 0.045, 0.33, 0.030])
586
587 LINTHRESH = 50 # [G] symlog linear band
588
589 # --- Helper: load arrays for a given (lcc, iccl) ---
590 def _get_arrays(lcc_v, iccl_v):
591     prof = filter_profile(df, lcc=lcc_v, iccl=iccl_v)
592     pos = prof[COL_POS].values
593     fld = prof[COL_FIELD].values
594     _, dBdz = field_derivative(pos, fld)
595     xings = find_zero_crossings(prof)
596     return pos, fld, dBdz, xings
597
598 # --- Initial draw ---
599 pos, fld, dBdz_arr, crossings = _get_arrays(
600     lcc_vals[lcc_idx], iccl_vals[iccl_idx])
601
602 def _title():
603     return (f"Longitudinal Field Profile & Gradient – interactive\n"
604            f"LCC = {lcc_vals[lcc_idx]:.0f} A | "
605            f"ICC1 = {iccl_vals[iccl_idx]:.3f} A | "
606            f"ICC2 = {iccl2_val:.3f} A")
607
608 fig.suptitle(_title(), fontsize=12)
609
610 # ---- top panel: Bz(z) ----
611 line_b, = ax_b.plot(pos, fld, color=LINE_COLOR, lw=LINE_WIDTH, zorder=3)
612 ax_b.axhline(0, **ZERO_LINE_KW)
613 ax_b.axhline(B_RESONANCE, color="darkorange", lw=1.0, ls="--", alpha=0.8,
614             label=f"±{B_RESONANCE:.0f} G")
615 ax_b.axhline(-B_RESONANCE, color="darkorange", lw=1.0, ls="--", alpha=0.8)
616
617 ax_b.set_yscale("symlog", linthresh=LINTHRESH, linscale=0.5)
618 ax_b.yaxis.set_minor_locator(
619     ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH,
620                                 subs=np.arange(2, 10)))
621 ax_b.set_ylabel("B$z$ (G) [symlog]", fontsize=11)
622 ax_b.tick_params(which="both", direction="in", top=True, right=True,
623                 labelbottom=False)
624 ax_b.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
625 ax_b.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
626 ax_b.legend(fontsize=9, loc="upper left")
627
628 # zero-crossing scatter (top panel)
629 sc_b = ax_b.scatter([], [], color="crimson", marker="o", s=60, zorder=5,
630                    label="Zero crossing")
631
632 # ---- bottom panel: dBz/dz(z) ----
633 # Use a Line2D with drawstyle="steps-post" – easier to update than
634 # ax.step() which returns a list and can't be set_data'd cleanly

```

```

635 line_d, = ax_d.plot(pos[:-1], dBdz_arr, color="darkorange",
636                     lw=LINE_WIDTH, drawstyle="steps-post", zorder=3)
637 ax_d.axhline(0, **ZERO_LINE_KW)
638 ax_d.set_ylabel(r"$dB_z/dz$ (G/cm)", fontsize=11)
639 ax_d.set_xlabel("Position (cm)", fontsize=11)
640 ax_d.tick_params(which="both", direction="in", top=True, right=True)
641 ax_d.xaxis.set_minor_locator(ticker.AutoMinorLocator())
642 ax_d.yaxis.set_minor_locator(ticker.AutoMinorLocator())
643 ax_d.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
644
645 # zero-crossing scatter + annotation (bottom panel)
646 sc_d = ax_d.scatter([], [], color="crimson", s=60, zorder=5)
647 ann_d = ax_d.annotate("", xy=(0, 0), fontsize=8, color="crimson",
648                       va="center")
649
650 # shared x limits (inverted: beam direction →14040)
651 ax_b.set_xlim(pos.max(), pos.min())
652
653 # — Vertical cursor —————
654 # Two axvline objects (one per panel) kept in sync via sharex, plus a
655 # text readout box in the top-right corner of the Bz panel.
656 vline_b = ax_b.axvline(x=np.nan, color="black", lw=1.0, ls="-",
657                       alpha=0.6, zorder=10)
658 vline_d = ax_d.axvline(x=np.nan, color="black", lw=1.0, ls="-",
659                       alpha=0.6, zorder=10)
660
661 readout = ax_b.text(
662     0.98, 0.97, "",
663     transform=ax_b.transAxes,
664     fontsize=9, va="top", ha="right",
665     fontfamily="monospace",
666     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
667             edgcolor="grey", alpha=0.85),
668     zorder=11
669 )
670
671 # — Crossing vlines (persistent, drawn once per profile load) —————
672 # Stored as a list so we can remove them when the profile changes.
673 _cross_vlines = []
674
675 def _draw_crossings(xings):
676     """Update crossing scatter markers; no persistent vlines."""
677     for vl in _cross_vlines:
678         try: vl.remove()
679         except ValueError: pass
680     _cross_vlines.clear()
681
682     if xings.empty():
683         sc_b.set_offsets(np.empty((0, 2)))
684         sc_d.set_offsets(np.empty((0, 2)))
685         ann_d.set_text("")
686         return
687
688     sc_b.set_offsets(
689         np.column_stack([xings["position_cm"].values,
690                         np.zeros(len(xings))]))
691     sc_d.set_offsets(
692         np.column_stack([xings["position_cm"].values,
693                         xings["dBdz_Gcm"].values]))
694
695     # annotation for the first (primary) crossing only
696     row0 = xings.iloc[0]
697     ann_d.set_text(f" {row0['dBdz_Gcm']:.2f} G/cm")
698     ann_d.xy = (row0["position_cm"], row0["dBdz_Gcm"])
699
700 _draw_crossings(crossings)
701
702 # — Interpolation helpers —————
703 def _interp_bz(x, pos_arr, fld_arr):
704     """Linear interpolation of Bz at position x."""
705     if x < min(pos_arr) or x > max(pos_arr):
706         return np.nan
707     return float(np.interp(x, pos_arr, fld_arr))

```

```

708 def _interp_dbdz(x, pos_arr, dBdz_a):
709     """
710     dBz/dz is piecewise-constant (step plot): return the value of the
711     interval that contains x, i.e. dBdz[i] where pos[i] <= x < pos[i+1].
712     """
713     if x < pos_arr[0] or x > pos_arr[-2]:
714         return np.nan
715     idx = int(np.searchsorted(pos_arr[:-1], x, side="right")) - 1
716     idx = max(0, min(idx, len(dBdz_a) - 1))
717     return float(dBdz_a[idx])
718
719 # — Mouse motion callback —————
720 def _on_mouse_move(event):
721     # Accept motion over either panel
722     if event.inaxes not in (ax_b, ax_d):
723         vline_b.set_xdata([np.nan])
724         vline_d.set_xdata([np.nan])
725         readout.set_text("")
726         fig.canvas.draw_idle()
727         return
728
729     x = event.xdata
730     if x is None:
731         return
732
733     vline_b.set_xdata([x])
734     vline_d.set_xdata([x])
735
736     bz = _interp_bz(x, pos, fld)
737     db = _interp_dbdz(x, pos, dBdz_arr)
738
739     bz_str = f"{bz:+.2f} G" if not np.isnan(bz) else "-"
740     db_str = f"{db:+.3f} G/cm" if not np.isnan(db) else "-"
741
742     readout.set_text(
743         f"z = {x:.2f} cm\n"
744         f"Bz = {bz_str}\n"
745         f"dBz/dz = {db_str}"
746     )
747     fig.canvas.draw_idle()
748
749 fig.canvas.mpl_connect("motion_notify_event", _on_mouse_move)
750
751 # — Profile redraw (called by both sliders) —————
752 def _redraw():
753     nonlocal pos, fld, dBdz_arr, crossings
754
755     pos, fld, dBdz_arr, crossings = _get_arrays(
756         lcc_vals[lcc_idx], iccl_vals[iccl_idx])
757
758     line_b.set_data(pos, fld)
759     line_d.set_data(pos[:-1], dBdz_arr)
760
761     # Rescale y-axes to new data (x is shared and inverted, unchanged)
762     ax_b.relim(); ax_b.autoscale_view(scalex=False)
763     ax_d.relim(); ax_d.autoscale_view(scalex=False)
764
765     _draw_crossings(crossings)
766     fig.suptitle(_title(), fontsize=12)
767     fig.canvas.draw_idle()
768
769 # — Sliders —————
770 sl_lcc = Slider(
771     ax_sl_lcc, "LCC (A)",
772     valmin=0, valmax=len(lcc_vals) - 1,
773     valinit=lcc_idx, valstep=1,
774     color="steelblue"
775 )
776
777 sl_iccl = Slider(
778     ax_sl_iccl, "ICCL (A)",
779     valmin=0, valmax=len(iccl_vals) - 1,
780     valinit=iccl_idx, valstep=1,

```



```

781     color="darkorange"
782 )
783
784 # Display actual physical value, not the index
785 def _fmt_lcc(v): return f"{lcc_vals[int(v)]:.0f}"
786 def _fmt_iccl(v): return f"{iccl_vals[int(v)]:.3f}"
787 sl_lcc.valtext.set_text(_fmt_lcc(lcc_idx))
788 sl_iccl.valtext.set_text(_fmt_iccl(iccl_idx))
789
790 def _on_lcc_change(val):
791     nonlocal lcc_idx
792     lcc_idx = int(round(val))
793     sl_lcc.valtext.set_text(_fmt_lcc(lcc_idx))
794     _redraw()
795
796 def _on_iccl_change(val):
797     nonlocal iccl_idx
798     iccl_idx = int(round(val))
799     sl_iccl.valtext.set_text(_fmt_iccl(iccl_idx))
800     _redraw()
801
802 sl_lcc.on_changed(_on_lcc_change)
803 sl_iccl.on_changed(_on_iccl_change)
804
805 # Keep slider references alive (prevents garbage collection)
806 fig_sliders = [sl_lcc, sl_iccl]
807
808 plt.show()
809
810
811
812 # =====
813 # FUNCTION 5c – plot_br_interactive
814 # =====
815
816 # Beam radii to show in the Br panel [cm]
817 BR_RADII_CM = [0.5, 1.0, 1.5]
818
819 # Colours for the three radii curves
820 BR_COLORS = ["steelblue", "darkorange", "seagreen"]
821
822 def plot_br_interactive(df: pd.DataFrame,
823                        lcc: float = DEFAULT_LCC,
824                        iccl: float = DEFAULT_ICCL) -> None:
825     """
826     Interactive two-panel figure – Bz(z) on top, Br(z) on the bottom.
827
828     Top panel : identical to plot_profile_interactive – Bz(z) on a symlog
829                  y-scale, zero-crossing scatter dots, shared vertical cursor.
830
831     Bottom panel: radial field component
832                    $Br(z) = -(dBz/dz) * r / 2$  [G]
833                   plotted as a step curve for three beam radii
834                    $r = 0.5, 1.0, 1.5$  cm (BR_RADII_CM)
835                   shown on a symlog y-scale so both the large values near the
836                   magnet and the small values near the crossing are readable.
837
838     Both panels share the x-axis (beam direction, 140 → 40 cm).
839
840     Cursor: hovering over either panel draws a shared vertical line and
841     shows a readout box in the top-right corner of the Bz panel:
842         z      = <position> cm
843         Bz     = <field> G
844         Br(0.5)= <value> G
845         Br(1.0)= <value> G
846         Br(1.5)= <value> G
847
848     Sliders: LCC and ICCL – same discrete stepping as plot_profile_interactive.
849
850     Requirements
851     -----
852     Requires an interactive matplotlib backend (Qt5 in Spyder).
853     """

```

```

854 from matplotlib.widgets import Slider
855 from matplotlib.gridspec import GridSpec
856
857 lcc_vals = sorted(df[COL_LCC].unique())
858 icc1_vals = sorted(df[COL_ICC1].unique())
859 icc2_val = df[COL_ICC2].iloc[0]
860
861 def _nearest_idx(val, arr):
862     return int(np.argmax(np.abs(np.array(arr) - val)))
863
864 lcc_idx = _nearest_idx(lcc, lcc_vals)
865 icc1_idx = _nearest_idx(icc1, icc1_vals)
866
867 # — Figure layout —————
868 # Rows: 0=Bz (tall), 1=Br (medium), 2=spacer, 3=sliders
869 fig = plt.figure(figsize=(11, 8.5))
870 gs = GridSpec(4, 1, figure=fig,
871             height_ratios=[2.6, 1.6, 0.15, 0.55],
872             hspace=0.08)
873
874 ax_b = fig.add_subplot(gs[0])
875 ax_r = fig.add_subplot(gs[1], sharex=ax_b)
876
877 fig.delaxes(fig.add_subplot(gs[3]))
878 ax_sl_lcc = fig.add_axes([0.12, 0.045, 0.33, 0.030])
879 ax_sl_icc1 = fig.add_axes([0.57, 0.045, 0.33, 0.030])
880
881 LINTHRESH_B = 50 # [G] symlog linear band for Bz
882 LINTHRESH_BR = 0.5 # [G] symlog linear band for Br (small near crossing)
883
884 # — Data helper —————
885 def _get_arrays(lcc_v, icc1_v):
886     prof = filter_profile(df, lcc=lcc_v, icc1=icc1_v)
887     pos = prof[COL_POS].values
888     fld = prof[COL_FIELD].values
889     mid, dBdz = field_derivative(pos, fld)
890     # Br[k] = -(dBz/dz) * r_k / 2, computed at midpoints
891     Br_list = [-dBdz * r / 2 for r in BR_RADII_CM]
892     xings = find_zero_crossings(prof)
893     return pos, fld, mid, dBdz, Br_list, xings
894
895 pos, fld, mid, dBdz_arr, Br_list, crossings = _get_arrays(
896     lcc_vals[lcc_idx], icc1_vals[icc1_idx])
897
898 def _title():
899     return (f"$B_z$ and $B_r$ vs Position – interactive\n"
900           f"LCC = {lcc_vals[lcc_idx]:.0f} A | "
901           f"ICC1 = {icc1_vals[icc1_idx]:.3f} A | "
902           f"ICC2 = {icc2_val:.3g} A")
903
904 fig.suptitle(_title(), fontsize=12)
905
906 # — Top panel: Bz(z) —————
907 line_b = ax_b.plot(pos, fld, color=LINE_COLOR, lw=LINE_WIDTH, zorder=3)
908 ax_b.axhline(0, **ZERO_LINE_KW)
909 ax_b.axhline(B_RESONANCE, color="darkorange", lw=1.0, ls="--",
910             alpha=0.8, label=f"±{B_RESONANCE:.0f} G")
911 ax_b.axhline(-B_RESONANCE, color="darkorange", lw=1.0, ls="--", alpha=0.8)
912
913 ax_b.set_yscale("symlog", linthresh=LINTHRESH_B, linscale=0.5)
914 ax_b.yaxis.set_minor_locator(
915     ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH_B,
916                                 subs=np.arange(2, 10)))
917 ax_b.set_ylabel(r"$B_z$ (G) [symlog]", fontsize=11)
918 ax_b.tick_params(which="both", direction="in", top=True, right=True,
919                 labelbottom=False)
920 ax_b.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
921 ax_b.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
922 ax_b.legend(fontsize=9, loc="upper left")
923
924 # — Bottom panel: Br(z) for each radius —————
925 # Use steps-post (same convention as dBdz): each interval spans pos[i→]pos[i+1]
926 lines_r = []

```

```

927 for k, (r, col, Br) in enumerate(zip(BR_RADII_CM, BR_COLORS, Br_list)):
928     ln, = ax_r.plot(mid, Br, color=col, lw=LINE_WIDTH,
929                     drawstyle="steps-mid", zorder=3,
930                     label=f"r = {r} cm")
931     lines_r.append(ln)
932
933 ax_r.axhline(0, **ZERO_LINE_KW)
934 ax_r.set_yscale("symlog", linthresh=LINTHRESH_BR, linscale=0.4)
935 ax_r.yaxis.set_minor_locator(
936     ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH_BR,
937                                 subs=np.arange(2, 10)))
938 ax_r.set_ylabel(r"$B_r$ (G) [symlog]", fontsize=11)
939 ax_r.set_xlabel("Position (cm)", fontsize=11)
940 ax_r.tick_params(which="both", direction="in", top=True, right=True)
941 ax_r.xaxis.set_minor_locator(ticker.AutoMinorLocator())
942 ax_r.yaxis.set_minor_locator(ticker.AutoMinorLocator())
943 ax_r.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
944 ax_r.legend(fontsize=9, loc="upper left",
945            title=r"$B_r = -\frac{dB_z}{dz}\backslash, \frac{r}{2}$",
946            title_fontsize=8)
947
948 ax_b.set_xlim(pos.max(), pos.min()) # beam direction: →14040 cm
949
950 # — Crossing scatter – Bz panel (dots at zero on field profile) —————
951 # — Crossing scatter – Br panel (dots at Br value for each radius) —————
952 # Each is a list of 3 scatter artists – one per radius curve.
953 # They are updated in-place by draw_crossings.
954 sc_b = ax_b.scatter([], [], color="crimson", marker="o", s=60, zorder=6)
955
956 sc_r_list = []
957 for col in BR_COLORS:
958     sc = ax_r.scatter([], [], s=55, zorder=6,
959                       edgecolors=col, facecolors="white", linewidths=1.5)
960     sc_r_list.append(sc)
961
962 def _draw_crossings(xings):
963     """Update crossing dots on both panels."""
964     if xings.empty:
965         sc_b.set_offsets(np.empty((0, 2)))
966         for sc in sc_r_list:
967             sc.set_offsets(np.empty((0, 2)))
968     return
969
970 # Bz panel: dots at (x0, 0) – same as before
971 sc_b.set_offsets(np.column_stack([xings["position_cm"].values,
972                                  np.zeros(len(xings))]))
973
974 # Br panel: for each radius, one dot per crossing at (x0, Br(x0))
975 # Br is piecewise-constant on midpoints – use nearest midpoint lookup
976 for sc, Br in zip(sc_r_list, Br_list):
977     pts = []
978     for x0 in xings["position_cm"].values:
979         idx = int(np.argmin(np.abs(mid - x0)))
980         idx = max(0, min(idx, len(Br) - 1))
981         pts.append([x0, Br[idx]])
982     sc.set_offsets(np.array(pts))
983
984 _draw_crossings(crossings)
985
986 # — Vertical cursor —————
987 vline_b = ax_b.axvline(x=np.nan, color="black", lw=1.0, ls="-",
988                        alpha=0.6, zorder=10)
989 vline_r = ax_r.axvline(x=np.nan, color="black", lw=1.0, ls="-",
990                        alpha=0.6, zorder=10)
991
992 readout = ax_b.text(
993     0.98, 0.97, "",
994     transform=ax_b.transAxes,
995     fontsize=8.5, va="top", ha="right",
996     fontfamily="monospace",
997     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
998             edgecolor="grey", alpha=0.87),
999     zorder=11

```

```

1000 )
1001
1002 # — Interpolation —————
1003 def _interp_bz(x):
1004     if x < pos.min() or x > pos.max():
1005         return np.nan
1006     return float(np.interp(x, pos, fld))
1007
1008 def _interp_br(x, Br):
1009     """Br is defined on midpoints (mid array); piecewise-constant steps."""
1010     if x < mid[0] or x > mid[-1]:
1011         return np.nan
1012     idx = int(np.searchsorted(mid, x, side="right")) - 1
1013     idx = max(0, min(idx, len(Br) - 1))
1014     return float(Br[idx])
1015
1016 # — Mouse motion —————
1017 def _on_move(event):
1018     if event.inaxes not in (ax_b, ax_r) or event.xdata is None:
1019         vline_b.set_xdata([np.nan])
1020         vline_r.set_xdata([np.nan])
1021         readout.set_text("")
1022         fig.canvas.draw_idle()
1023     return
1024
1025     x = event.xdata
1026     vline_b.set_xdata([x])
1027     vline_r.set_xdata([x])
1028
1029     bz = _interp_bz(x)
1030     br_vals = [_interp_br(x, Br) for Br in Br_list]
1031
1032     bz_str = f"{bz:+.2f} G" if not np.isnan(bz) else "-"
1033     br_strs = [f"{v:+.3f} G" if not np.isnan(v) else "-"
1034               for v in br_vals]
1035
1036     lines = [f"z      = {x:.2f} cm",
1037             f"Bz      = {bz_str}"]
1038     for r, bs in zip(BR_RADII_CM, br_strs):
1039         lines.append(f"Br({r:.1f}) = {bs}")
1040     readout.set_text("\n".join(lines))
1041     fig.canvas.draw_idle()
1042
1043 fig.canvas.mpl_connect("motion_notify_event", _on_move)
1044
1045 # — Redraw on slider change —————
1046 def _redraw():
1047     nonlocal pos, fld, mid, dBdz_arr, Br_list, crossings
1048
1049     pos, fld, mid, dBdz_arr, Br_list, crossings = _get_arrays(
1050         lcc_vals[lcc_idx], iccl_vals[iccl_idx])
1051
1052     line_b.set_data(pos, fld)
1053     for ln, Br in zip(lines_r, Br_list):
1054         ln.set_data(mid, Br)
1055
1056     ax_b.relim(); ax_b.autoscale_view(scalex=False)
1057     ax_r.relim(); ax_r.autoscale_view(scalex=False)
1058
1059     _draw_crossings(crossings)
1060     fig.suptitle(_title(), fontsize=12)
1061     fig.canvas.draw_idle()
1062
1063 # — Sliders —————
1064 sl_lcc = Slider(ax_sl_lcc, "LCC (A)",
1065                valmin=0, valmax=len(lcc_vals) - 1,
1066                valinit=lcc_idx, valstep=1, color="steelblue")
1067 sl_iccl = Slider(ax_sl_iccl, "ICCL (A)",
1068                valmin=0, valmax=len(iccl_vals) - 1,
1069                valinit=iccl_idx, valstep=1, color="darkorange")
1070
1071 def _fmt_lcc(v): return f"{lcc_vals[int(v)]:.0f}"
1072 def _fmt_iccl(v): return f"{iccl_vals[int(v)]:.3f}"

```

```

1073 sl_lcc.valtext.set_text(_fmt_lcc(lcc_idx))
1074 sl_iccl.valtext.set_text(_fmt_iccl(iccl_idx))
1075
1076 def _on_lcc(val):
1077     nonlocal lcc_idx
1078     lcc_idx = int(round(val))
1079     sl_lcc.valtext.set_text(_fmt_lcc(lcc_idx))
1080     _redraw()
1081
1082 def _on_iccl(val):
1083     nonlocal iccl_idx
1084     iccl_idx = int(round(val))
1085     sl_iccl.valtext.set_text(_fmt_iccl(iccl_idx))
1086     _redraw()
1087
1088 sl_lcc.on_changed(_on_lcc)
1089 sl_iccl.on_changed(_on_iccl)
1090 fig._sliders = [sl_lcc, sl_iccl]
1091
1092 plt.show()
1093
1094
1095
1096
1097 def plot_summary_grid(scan: pd.DataFrame,
1098                      sona: dict = None) -> None:
1099     """
1100     Summary grid figure: rows = ICC1 values, 2 columns per row.
1101
1102     Column 0 – zero-crossing position  $x$  [cm] vs LCC
1103     Column 1 – field gradient dBz/dz [G/cm] vs LCC
1104
1105     Every individual crossing is plotted – not just a single "primary".
1106     When a condition has  $N$  crossings (e.g. 3), all  $N$  points appear at that
1107     LCC value, each connected to its counterpart at neighbouring LCC values
1108     by a line (crossing-index 1 connected to crossing-index 1, etc.).
1109
1110     Crossing series are distinguished by:
1111         1st crossing – circle ● steelblue (solid line)
1112         2nd crossing – square ■ darkorange (dashed line)
1113         3rd crossing – diamond ♦ seagreen (dotted line)
1114         4th+ – plus + grey (dotted line)
1115
1116     Where a crossing index does not exist for a particular LCC, the line
1117     is simply interrupted – no imputation, no arbitrary selection.
1118
1119     Parameters
1120     -----
1121     scan : pd.DataFrame – output of scan_conditions()
1122     sona : dict or None – output of compute_sona_parameters()
1123     """
1124     iccl_vals = sorted(scan["ICC1"].unique())
1125     lcc_vals = sorted(scan["LCC"].unique())
1126     n_iccl = len(iccl_vals)
1127     icc2_val = scan["ICC2"].iloc[0]
1128
1129     # — Find the maximum number of crossings across all conditions —————
1130     max_crossings = max(len(r["all_crossings"]) for _, r in scan.iterrows())
1131
1132     # — Build per-crossing arrays: for each ICC1, for each crossing index,
1133     # collect (LCC,  $x_0$ , dBdz) only where that index exists —————
1134     # Structure: crossing_data[iccl_idx][crossing_idx] = [(lcc,  $x_0$ , dBdz), ...]
1135     crossing_data = []
1136     for iccl in iccl_vals:
1137         sub = scan[np.abs(scan["ICC1"] - iccl) < 1e-6].sort_values("LCC")
1138         per_index = [[] for _ in range(max_crossings)]
1139         for _, row in sub.iterrows():
1140             for k, (x0, dBdz) in enumerate(row["all_crossings"]):
1141                 per_index[k].append((row["LCC"], x0, dBdz))
1142             crossing_data.append(per_index)
1143
1144     # — Figure —————
1145     fig, axes = plt.subplots(

```

```

1146     n_iccl1, 2,
1147     figsize=(13, 3.5 * n_iccl1),
1148     sharex=True,
1149     layout="constrained",
1150     gridspec_kw={"hspace": 0.10, "wspace": 0.30}
1151 )
1152 axes = np.atleast_2d(axes)
1153
1154 fig.suptitle(
1155     f"Zero-Crossing Summary - ICC2 = {icc2_val:.3g} A\n"
1156     f"Left: crossing position $x_0$ [cm] | "
1157     f"Right: gradient $dB_z/dz$ at crossing [G/cm] | "
1158     f"each series = one crossing index",
1159     fontsize=12, y=1.01
1160 )
1161
1162 for row_idx, iccl1 in enumerate(icc1_vals):
1163     ax_x0 = axes[row_idx, 0]
1164     ax_dBdz = axes[row_idx, 1]
1165     per_index = crossing_data[row_idx]
1166
1167     legend_handles = []
1168
1169     for k, pts in enumerate(per_index):
1170         if not pts:
1171             continue
1172         style = SERIES_STYLE[min(k, len(SERIES_STYLE) - 1)]
1173         lcc_k = [p[0] for p in pts]
1174         x0_k = [p[1] for p in pts]
1175         dBdz_k = [p[2] for p in pts]
1176
1177         h1, = ax_x0.plot(lcc_k, x0_k,
1178                         color=style["color"], marker=style["marker"],
1179                         markersize=style["ms"], linewidth=style["lw"],
1180                         linestyle=style["ls"], zorder=3 + k,
1181                         label=style["label"])
1182         ax_dBdz.plot(lcc_k, dBdz_k,
1183                     color=style["color"], marker=style["marker"],
1184                     markersize=style["ms"], linewidth=style["lw"],
1185                     linestyle=style["ls"], zorder=3 + k)
1186         legend_handles.append(h1)
1187
1188     # Sona threshold on the dBdz panel
1189     ax_dBdz.axhline(0, **ZERO_LINE_KW)
1190     if sona is not None:
1191         thr = sona["dBdz_sona_Gcm"]
1192         ax_dBdz.axhline(thr, color="crimson", lw=1.0, ls="--", alpha=0.8,
1193                        label=f"±{thr:.1f} G/cm (Sona)")
1194         ax_dBdz.axhline(-thr, color="crimson", lw=1.0, ls="--", alpha=0.8)
1195
1196     # Legend on the position panel (crossing series) and dBdz panel (Sona)
1197     if legend_handles:
1198         ax_x0.legend(handles=legend_handles, fontsize=7, loc="best",
1199                     title="crossing", title_fontsize=7)
1200     if sona is not None:
1201         ax_dBdz.legend(fontsize=7, loc="best")
1202
1203     ax_x0.set_ylabel(f"ICC1 = {iccl1:.3f} A\n$x_0$ (cm)", fontsize=9)
1204     ax_x0.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1205     ax_x0.tick_params(which="both", direction="in", top=True, right=True)
1206     ax_x0.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1207
1208     ax_dBdz.set_ylabel(r"$dB_z/dz$ (G/cm)", fontsize=9)
1209     ax_dBdz.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1210     ax_dBdz.tick_params(which="both", direction="in", top=True, right=True)
1211     ax_dBdz.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1212
1213     # shared x-axis label + ticks on the bottom row only
1214     axes[-1, 0].set_xlabel("LCC (A)", fontsize=10)
1215     axes[-1, 1].set_xlabel("LCC (A)", fontsize=10)
1216     # only label every other LCC tick if there are many
1217     if len(lcc_vals) > 20:
1218         tick_step = 4

```

```

1219     axes[-1, 0].set_xticks(lcc_vals[:, :tick_step])
1220     axes[-1, 1].set_xticks(lcc_vals[:, :tick_step])
1221 else:
1222     axes[-1, 0].set_xticks(lcc_vals)
1223     axes[-1, 1].set_xticks(lcc_vals)
1224
1225 # column titles on the top row
1226 axes[0, 0].set_title("Zero-crossing position  $x_0$",  fontsize=11, pad=6)
1227 axes[0, 1].set_title(r"Gradient  $dB_z/dz$ at crossing",  fontsize=11, pad=6)
1228
1229 if SAVE_FIGURES:
1230     fname = f"summary_grid_ICC2_{icc2_val:.3g}.png"
1231     fig.savefig(fname, dpi=130, bbox_inches="tight")
1232     print(f"[plot_summary_grid]  Saved {fname}")
1233 plt.show()
1234
1235
1236 # =====
1237 # FUNCTION 7 – plot_profiles_grid  (Fig 1a)
1238 # =====
1239
1240 def plot_profiles_grid(df: pd.DataFrame,
1241                      scan: pd.DataFrame = None) -> None:
1242     """
1243     Fig 1a – Bz(z) profile grid.
1244
1245     Layout
1246     -----
1247     Rows      = ICC1 values  (one row per ICC1)
1248     Columns   = 1  (single column of panels)
1249     Lines     = one per LCC value, colour-mapped via viridis
1250
1251     Each panel shows the full longitudinal field profile on a symlog y-scale
1252     so both the small-field zero-crossing region and the large positive tail
1253     are simultaneously visible.
1254
1255     Additional decorations per panel
1256     -----
1257     • Horizontal dashed lines at 0 G and  $\pm B_{\text{RESONANCE}}$  G
1258     • Vertical dashed lines at the primary zero-crossing position for
1259       every LCC value (thin, same colour as the profile line)
1260     • Red triangle marker  $\blacktriangle()$  on the zero line when  $n_{\text{crossings}} > 1$ 
1261
1262     X-axis is inverted (140  $\rightarrow$  40 cm) to match beam-line convention.
1263
1264     Parameters
1265     -----
1266     df      : output of load_data()
1267     scan    : optional output of scan_conditions(); if supplied, crossing
1268               positions are marked with vertical dashed lines
1269     """
1270     icc1_vals = sorted(df[COL_ICC1].unique())
1271     lcc_vals  = sorted(df[COL_LCC].unique())
1272     n_icc1    = len(icc1_vals)
1273     n_lcc     = len(lcc_vals)
1274     icc2_val  = df[COL_ICC2].iloc[0]
1275
1276     CMAP_LCC = plt.cm.viridis
1277     lcc_colors = {v: CMAP_LCC(i / max(n_lcc - 1, 1))
1278                  for i, v in enumerate(lcc_vals)}
1279
1280     LINTHRESH = 50  # [G] linear band around zero on symlog scale
1281
1282     fig, axes = plt.subplots(
1283         n_icc1, 1,
1284         figsize=(10, 3.8 * n_icc1),
1285         sharey=True,
1286         layout="constrained"
1287     )
1288     axes = np.atleast_1d(axes)
1289
1290     fig.suptitle(
1291         f"Fig 1a – Longitudinal Field Profiles  $B_z(z)$\n"

```



```

1292     f"ICC2 = {icc2_val:.3g} A | colour = LCC [A] | "
1293     f"x-axis: beam direction 140 → 40 cm",
1294     fontsize=13
1295 )
1296
1297 pos_min = df[COL_POS].min()
1298 pos_max = df[COL_POS].max()
1299
1300 for row_idx, icc1 in enumerate(icc1_vals):
1301     ax = axes[row_idx]
1302
1303     for lcc in lcc_vals:
1304         mask = ((np.abs(df[COL_LCC] - lcc) < 1e-6) &
1305                 (np.abs(df[COL_ICC1] - icc1) < 1e-6))
1306         prof = df[mask].sort_values(COL_POS)
1307         if prof.empty:
1308             continue
1309
1310         col = lcc_colors[lcc]
1311         ax.plot(prof[COL_POS], prof[COL_FIELD],
1312                 color=col, linewidth=1.6, zorder=3,
1313                 label=f"{lcc:.0f} A")
1314
1315         # vertical lines at every crossing (if scan provided)
1316         # colour matches SERIES_STYLE: 1st=steelblue, 2nd=orange, 3rd=seagreen
1317         if scan is not None:
1318             scan_row = scan[(np.abs(scan["LCC"] - lcc) < 1e-6) &
1319                             (np.abs(scan["ICC1"] - icc1) < 1e-6)]
1320             if not scan_row.empty:
1321                 for k, (x0, _) in enumerate(scan_row.iloc[0]["all_crossings"]):
1322                     st = SERIES_STYLE[min(k, len(SERIES_STYLE) - 1)]
1323                     ax.axvline(x0, color=st["color"], lw=1.0, ls="--",
1324                               alpha=0.7, zorder=5)
1325
1326         # reference lines
1327         ax.axhline(0, **ZERO_LINE_KW)
1328         ax.axhline(B_RESONANCE, color="darkorange", lw=1.0, ls="--",
1329                   alpha=0.8)
1330         ax.axhline(-B_RESONANCE, color="darkorange", lw=1.0, ls="--",
1331                   alpha=0.8, label=f"±{B_RESONANCE:.0f} G")
1332
1333         ax.set_yscale("symlog", linthresh=LINTHRESH, linscale=0.5)
1334         ax.yaxis.set_minor_locator(
1335             ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH,
1336                                          subs=np.arange(2, 10)))
1337         ax.set_xlim(pos_max, pos_min) # beam direction
1338         # Row identifier on the right spine – keeps left y-axis clean
1339         ax.set_ylabel(f"ICC1 = {icc1:.3f} A", fontsize=9)
1340         ax.tick_params(which="both", direction="in", top=True, right=True)
1341         ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1342         ax.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
1343         # No legend – LCC is already encoded by the colorbar
1344
1345         axes[-1].set_xlabel("Position (cm)", fontsize=10)
1346         fig.supylabel(r"$B_z$ (G) [symlog]", fontsize=10)
1347         sm = plt.cm.ScalarMappable(
1348             cmap=CMAP_LCC,
1349             norm=plt.Normalize(vmin=min(lcc_vals), vmax=max(lcc_vals)))
1350         sm.set_array([])
1351         fig.colorbar(sm, ax=axes.tolist(), label="LCC (A)", shrink=0.6, pad=0.02)
1352
1353     if SAVE_FIGURES:
1354         fname = f"figla_profiles_ICC2_{icc2_val:.3g}.png"
1355         fig.savefig(fname, dpi=130, bbox_inches="tight")
1356         print(f"[plot_profiles_grid] Saved {fname}")
1357     plt.show()
1358     return axes
1359
1360 # =====
1361 # FUNCTION 8 – plot_gradients_grid (Fig 1b)
1362 # =====
1363
1364

```

```

1365 def plot_gradients_grid(df: pd.DataFrame,
1366                         scan: pd.DataFrame = None,
1367                         sona: dict = None) -> None:
1368     """
1369     Fig 1b – dBz/dz(z) gradient grid.
1370
1371     Identical panel layout to Fig 1a (rows = ICC1, colour = LCC) but the
1372     y-axis shows the forward finite-difference gradient dBz/dz [G/cm].
1373
1374     The gradient spans roughly -30 to +1500 G/cm, so a symlog scale is used
1375     (linear band ±LINTHRESH around zero, log elsewhere) to keep the
1376     zero-crossing region readable alongside the large values near the magnet.
1377
1378     Additional decorations
1379     -----
1380     • Dot at the primary zero-crossing position (0x, dBdz) – same colour
1381       as the profile line, black edge – when scan is provided
1382     • Vertical dotted line at 0x (thin, same colour)
1383     • Horizontal dashed lines at ±sona['dBdz_sona_Gcm'] when sona is provided
1384
1385     Parameters
1386     -----
1387     df : output of load_data()
1388     scan : optional output of scan_conditions()
1389     sona : optional output of compute_sona_parameters()
1390     """
1391     icc1_vals = sorted(df[COL_ICC1].unique())
1392     lcc_vals = sorted(df[COL_LCC].unique())
1393     n_icc1 = len(icc1_vals)
1394     n_lcc = len(lcc_vals)
1395     icc2_val = df[COL_ICC2].iloc[0]
1396
1397     CMAP_LCC = plt.cm.viridis
1398     lcc_colors = {v: CMAP_LCC(i / max(n_lcc - 1, 1))
1399                  for i, v in enumerate(lcc_vals)}
1400
1401     LINTHRESH_D = 5.0 # [G/cm] linear band for the gradient symlog scale
1402
1403     fig, axes = plt.subplots(
1404         n_icc1, 1,
1405         figsize=(10, 3.8 * n_icc1),
1406         sharey=True,
1407         layout="constrained"
1408     )
1409     axes = np.atleast_1d(axes)
1410
1411     sona_label = (f"±{sona['dBdz_sona_Gcm']:.1f} G/cm (Sona)"
1412                  if sona is not None else "")
1413     fig.suptitle(
1414         f"Fig 1b – Longitudinal Field Gradient $dB_z/dz$\n"
1415         f"ICC2 = {icc2_val:.3g} A | colour = LCC [A] | "
1416         + (f"crimson dashes = {sona_label}" if sona else ""),
1417         fontsize=13
1418     )
1419
1420     pos_min = df[COL_POS].min()
1421     pos_max = df[COL_POS].max()
1422
1423     for row_idx, icc1 in enumerate(icc1_vals):
1424         ax = axes[row_idx]
1425
1426         for lcc in lcc_vals:
1427             mask = ((np.abs(df[COL_LCC] - lcc) < 1e-6) &
1428                    (np.abs(df[COL_ICC1] - icc1) < 1e-6))
1429             prof = df[mask].sort_values(COL_POS).reset_index(drop=True)
1430             if prof.empty:
1431                 continue
1432
1433             pos_d = prof[COL_POS].values
1434             fld_d = prof[COL_FIELD].values
1435             _, dBdz_arr = field_derivative(pos_d, fld_d)
1436             col = lcc_colors[lcc]
1437

```

```

1438     # Step plot: each dBdz[i] spans [pos[i], pos[i+1]]
1439     ax.step(pos_d[:-1], dBdz_arr, where="post",
1440            color=col, linewidth=1.5, zorder=3,
1441            label=f"{lcc:.0f} A")
1442     ax.hlines(dBdz_arr[-1], pos_d[-2], pos_d[-1],
1443            colors=col, linewidth=1.5, zorder=3)
1444
1445     # dot + vertical at every crossing (colour = SERIES_STYLE index)
1446     if scan is not None:
1447         scan_row = scan[(np.abs(scan["LCC"] - lcc) < 1e-6) &
1448                        (np.abs(scan["ICC1"] - icc1) < 1e-6)]
1449         if not scan_row.empty:
1450             for k, (x0, dBdz_c) in enumerate(scan_row.iloc[0]["all_crossings"]):
1451                 st = SERIES_STYLE[min(k, len(SERIES_STYLE) - 1)]
1452                 # vline colour = crossing index (series), so 1st/2nd/3rd are distinct
1453                 ax.axvline(x0, color=st["color"], lw=1.0, ls="--",
1454                        alpha=0.7, zorder=5)
1455                 # dot colour = LCC viridis colour, so you can identify which LCC
1456                 ax.scatter(x0, dBdz_c, color=col, s=55,
1457                        zorder=6, edgecolors=st["color"],
1458                        linewidths=1.2)
1459
1460     # reference lines
1461     ax.axhline(0, **ZERO_LINE_KW)
1462     if sona is not None:
1463         thr = sona["dBdz_sona_Gcm"]
1464         ax.axhline(thr, color="crimson", lw=1.0, ls="--", alpha=0.85)
1465         ax.axhline(-thr, color="crimson", lw=1.0, ls="--", alpha=0.85)
1466
1467     ax.set_yscale("symlog", linthresh=LINTHRESH_D, linscale=0.4)
1468     ax.yaxis.set_minor_locator(
1469         ticker.SymmetricalLogLocator(base=10, linthresh=LINTHRESH_D,
1470                                     subs=np.arange(2, 10)))
1471     ax.set_xlim(pos_max, pos_min)
1472     ax.set_ylabel(f"ICC1 = {icc1:.3f} A", fontsize=9)
1473     ax.tick_params(which="both", direction="in", top=True, right=True)
1474     ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1475     ax.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
1476     # No legend – LCC is already encoded by the colorbar
1477
1478     axes[-1].set_xlabel("Position (cm)", fontsize=10)
1479     fig.supylabel(r"$dB_z/dz$ (G/cm) [symlog]", fontsize=10)
1480
1481     sm = plt.cm.ScalarMappable(
1482         cmap=CMAP_LCC,
1483         norm=plt.Normalize(vmin=min(lcc_vals), vmax=max(lcc_vals)))
1484     sm.set_array([])
1485     fig.colorbar(sm, ax=axes.tolist(), label="LCC (A)", shrink=0.6, pad=0.02)
1486
1487     if SAVE_FIGURES:
1488         fname = f"fig1b_gradients_ICC2_{icc2_val:.3g}.png"
1489         fig.savefig(fname, dpi=130, bbox_inches="tight")
1490         print(f"[plot_gradients_grid] Saved {fname}")
1491     plt.show()
1492     return axes
1493
1494
1495 # =====
1496 # FUNCTION 9 – link_fig1_axes
1497 # =====
1498
1499 def link_fig1_axes(axes_1a: np.ndarray,
1500                   axes_1b: np.ndarray) -> None:
1501     """
1502     Link axes across and within Fig 1a and Fig 1b.
1503
1504     X-axis (Position cm) – linked across ALL panels in BOTH figures:
1505     zooming/panning any panel's x updates every other panel in both
1506     figures simultaneously.
1507
1508     Y-axis – linked WITHIN each figure separately:
1509     • Fig 1a panels share the same Bz (G) symlog scale
1510     • Fig 1b panels share the same dBz/dz (G/cm) symlog scale

```

```

1511     Y-axes are NOT linked between figures (different units).
1512
1513     Note: intra-figure sharey=True at plt.subplots time already links
1514     y-limits for panels within the same figure. The ylim_changed
1515     callbacks here are therefore redundant for the common case but
1516     serve as a belt-and-suspenders guard if sharey ever gets broken
1517     (e.g. by ax.twinx() usage elsewhere).
1518
1519     Parameters
1520     -----
1521     axes_1a : ndarray of Axes from plot_profiles_grid (shape: n_icc1,)
1522     axes_1b : ndarray of Axes from plot_gradients_grid (shape: n_icc1,)
1523
1524     Usage
1525     -----
1526     >>> axes_a = plot_profiles_grid(df, scan=scan)
1527     >>> axes_b = plot_gradients_grid(df, scan=scan, sona=sona)
1528     >>> link_fig1_axes(axes_a, axes_b)
1529     """
1530     all_axes = list(axes_1a.ravel()) + list(axes_1b.ravel())
1531     axes_a = list(axes_1a.ravel())
1532     axes_b = list(axes_1b.ravel())
1533
1534     _busy = [False] # re-entrancy guard (shared across all callbacks)
1535
1536     # — x-axis: all panels in both figures —————
1537     def _on_xlim_changed(src):
1538         if _busy[0]:
1539             return
1540         _busy[0] = True
1541         try:
1542             lim = src.get_xlim()
1543             for ax in all_axes:
1544                 if ax is not src:
1545                     ax.set_xlim(lim)
1546             for fig in {ax.get_figure() for ax in all_axes}:
1547                 fig.canvas.draw_idle()
1548         finally:
1549             _busy[0] = False
1550
1551     # — y-axis: within Fig 1a only —————
1552     def _on_ylim_changed_a(src):
1553         if _busy[0]:
1554             return
1555         _busy[0] = True
1556         try:
1557             lim = src.get_ylim()
1558             for ax in axes_a:
1559                 if ax is not src:
1560                     ax.set_ylim(lim)
1561             src.get_figure().canvas.draw_idle()
1562         finally:
1563             _busy[0] = False
1564
1565     # — y-axis: within Fig 1b only —————
1566     def _on_ylim_changed_b(src):
1567         if _busy[0]:
1568             return
1569         _busy[0] = True
1570         try:
1571             lim = src.get_ylim()
1572             for ax in axes_b:
1573                 if ax is not src:
1574                     ax.set_ylim(lim)
1575             src.get_figure().canvas.draw_idle()
1576         finally:
1577             _busy[0] = False
1578
1579     for ax in all_axes:
1580         ax.callbacks.connect("xlim_changed", _on_xlim_changed)
1581     for ax in axes_a:
1582         ax.callbacks.connect("ylim_changed", _on_ylim_changed_a)
1583     for ax in axes_b:

```

```

1584     ax.callbacks.connect("ylim_changed", _on_ylim_changed_b)
1585
1586     print(f"[link_fig1_axes] x linked: {len(all_axes)} axes (both figures) | "
1587           f"y linked: {len(axes_a)} in Fig1a, {len(axes_b)} in Fig1b")
1588
1589
1590 # =====
1591 # FUNCTION 10 – load_polarization
1592 # =====
1593
1594 def load_polarization(path: str = POL_DATA_PATH) -> pd.DataFrame:
1595     """
1596     Load the beam polarization scan CSV.
1597
1598     The file contains one row per LCC setting with 20 individual polarization
1599     measurements stored as a space-separated string in the 'POL. VALUES' column.
1600     This function parses those values and returns a clean DataFrame with:
1601
1602         LCC      [A] – coil current (from 'SET AVG.' column)
1603         pol_mean [%] – mean of the 20 measurements (== 'POL. AVG.')
1604         pol_sem  [%] – standard error of the mean (std / sqrt(n))
1605         pol_std  [%] – standard deviation
1606         n_meas   – number of individual measurements (typically 20)
1607
1608     Parameters
1609     -----
1610     path : str    Path to the polarization CSV file.
1611
1612     Returns
1613     -----
1614     pd.DataFrame sorted ascending by LCC.
1615     """
1616     raw = pd.read_csv(path)
1617
1618     records = []
1619     for _, row in raw.iterrows():
1620         lcc = float(row["SET AVG."])
1621         pol_avg = float(row["POL. AVG."])
1622         vals = [float(v) for v in str(row["POL. VALUES"]).split()]
1623         n = len(vals)
1624         pol_std = float(np.std(vals, ddof=1)) if n > 1 else 0.0
1625         pol_sem = pol_std / np.sqrt(n) if n > 0 else 0.0
1626         records.append(dict(LCC = lcc,
1627                             pol_mean = pol_avg,
1628                             pol_sem = pol_sem,
1629                             pol_std = pol_std,
1630                             n_meas = n))
1631
1632     df_pol = pd.DataFrame(records).sort_values("LCC").reset_index(drop=True)
1633
1634     print(f"[load_polarization] {len(df_pol)} LCC points "
1635           f"| LCC {df_pol['LCC'].min():.0f} → {df_pol['LCC'].max():.0f} A "
1636           f"| pol {df_pol['pol_mean'].min():.1f} – {df_pol['pol_mean'].max():.1f} %")
1637     return df_pol
1638
1639
1640 # =====
1641 # FUNCTION 11 – plot_polarization
1642 # =====
1643
1644 def plot_polarization(df_pol: pd.DataFrame) -> None:
1645     """
1646     Plot beam polarization (%) vs LCC (A).
1647
1648     Shows the mean polarization with ±1 SEM error bars and a ±1 std
1649     shaded band to convey measurement-to-measurement spread.
1650     """
1651     fig, ax = plt.subplots(figsize=(12, 5), layout="constrained")
1652
1653     lcc = df_pol["LCC"].values
1654     pol = df_pol["pol_mean"].values
1655     sem = df_pol["pol_sem"].values
1656     std = df_pol["pol_std"].values

```

```

1657
1658 # — Shaded  $\pm 1$  std band —————
1659 ax.fill_between(lcc, pol - std, pol + std,
1660                color="steelblue", alpha=0.15, label=" $\pm 1$  std")
1661
1662 # — Mean  $\pm$  SEM error bars —————
1663 ax.errorbar(lcc, pol, yerr=sem,
1664            fmt="o-", color="steelblue", markersize=4,
1665            linewidth=1.4, elinewidth=1.0, capsize=3,
1666            label="mean  $\pm$  SEM", zorder=4)
1667
1668 ax.set_xlabel("LCC (A)", fontsize=11)
1669 ax.set_ylabel("Beam polarization (%)", fontsize=11)
1670 ax.set_title(
1671     f"Beam Polarization vs LCC\n"
1672     f"ICC1 = {POL_ICC1:.3f} A | ICC2 = {POL_ICC2:.3g} A | "
1673     f"file: {POL_DATA_FILE}",
1674     fontsize=12
1675 )
1676 ax.tick_params(which="both", direction="in", top=True, right=True)
1677 ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
1678 ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1679 ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1680 ax.grid(True, which="minor", ls=":", lw=0.2, alpha=0.4)
1681 ax.legend(fontsize=9, loc="best")
1682
1683 if SAVE_FIGURES:
1684     fname = f"polarization_vs_LCC_ICC1_{POL_ICC1:.3f}.png"
1685     fig.savefig(fname, dpi=130, bbox_inches="tight")
1686     print(f"[plot_polarization] Saved {fname}")
1687 plt.show()
1688
1689
1690 # =====
1691 # HELPER – merge_pol_scan
1692 # =====
1693
1694 def merge_pol_scan(df_pol: pd.DataFrame,
1695                  scan: pd.DataFrame,
1696                  icc1: float = POL_ICC1) -> pd.DataFrame:
1697     """
1698     Merge the polarization DataFrame with the magnetic scan results for a
1699     given ICC1 value. Only LCC values present in both datasets are kept.
1700
1701     Returns a DataFrame with columns:
1702         LCC, pol_mean, pol_sem, pol_std,
1703         n_crossings, all_crossings,
1704         x0_1, dBdz_1, (1st crossing – highest z)
1705         x0_2, dBdz_2, (2nd crossing – if present, else NaN)
1706         x0_3, dBdz_3 (3rd crossing – if present, else NaN)
1707     """
1708     sub = scan[np.abs(scan["ICC1"] - icc1) < 1e-6].copy()
1709     merged = sub.merge(df_pol[["LCC", "pol_mean", "pol_sem", "pol_std"]],
1710                      on="LCC", how="inner")
1711
1712     for k, label in enumerate(["1", "2", "3"]):
1713         merged[f"x0_{label}"] = merged["all_crossings"].apply(
1714             lambda c: c[k][0] if len(c) > k else np.nan)
1715         merged[f"dBdz_{label}"] = merged["all_crossings"].apply(
1716             lambda c: c[k][1] if len(c) > k else np.nan)
1717
1718     merged = merged.sort_values("LCC").reset_index(drop=True)
1719     print(f"[merge_pol_scan] {len(merged)} matched LCC points "
1720           f"| ICC1={icc1:.3f} "
1721           f"| {(merged['n_crossings']==3).sum()} with 3 crossings, "
1722           f"| {(merged['n_crossings']==1).sum()} with 1")
1723     return merged
1724
1725
1726 # =====
1727 # FUNCTION 12 – plot_pol_lcc_sweep (Plot A)
1728 # =====
1729

```

```

1730 def plot_pol_lcc_sweep(merged: pd.DataFrame) -> None:
1731     """
1732     Plot A – Three-panel LCC sweep.
1733
1734     Panel 1 : beam polarization [%] vs LCC
1735     Panel 2 : zero-crossing positions  $x$  [cm] vs LCC (one series per crossing index)
1736     Panel 3 : dBdz [G/cm] at each crossing vs LCC (one series per crossing index)
1737
1738     All three panels share the same x-axis (LCC).
1739     Crossing series use SERIES_STYLE colours (steelblue / orange / seagreen).
1740     Points that don't exist (e.g. 2nd/3rd crossing for single-crossing LCC
1741     values) are simply absent – no imputation.
1742     """
1743     lcc = merged["LCC"].values
1744
1745     fig, (ax_pol, ax_x0, ax_db) = plt.subplots(
1746         3, 1, figsize=(12, 9),
1747         sharex=True, layout="constrained",
1748         gridspec_kw={"height_ratios": [1.4, 1, 1], "hspace": 0.06}
1749     )
1750     fig.suptitle(
1751         f"Beam Polarization & Zero-Crossing Properties vs LCC\n"
1752         f"ICC1 = {POL_ICC1:.3f} A | ICC2 = {POL_ICC2:.3g} A",
1753         fontsize=12
1754     )
1755
1756     # — Panel 1: polarization —————
1757     ax_pol.fill_between(lcc,
1758                        merged["pol_mean"] - merged["pol_std"],
1759                        merged["pol_mean"] + merged["pol_std"],
1760                        color="steelblue", alpha=0.15, label="±1 std")
1761     ax_pol.errorbar(lcc, merged["pol_mean"], yerr=merged["pol_sem"],
1762                    fmt="o-", color="steelblue", markersize=4,
1763                    lw=1.5, elinewidth=1.0, capsize=3,
1764                    label="mean ± SEM", zorder=4)
1765     ax_pol.set_ylabel("Polarization (%)", fontsize=10)
1766     ax_pol.legend(fontsize=8, loc="upper right")
1767     ax_pol.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1768     ax_pol.tick_params(which="both", direction="in", top=True, right=True)
1769     ax_pol.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1770
1771     # — Panels 2 & 3: crossing properties —————
1772     for k, label in enumerate(["1", "2", "3"]):
1773         st = SERIES_STYLE[k]
1774         x0_k = merged[f"x0_{label}"].values
1775         db_k = merged[f"dBdz_{label}"].values
1776         mask = ~np.isnan(x0_k)
1777
1778         ax_x0.plot(lcc[mask], x0_k[mask],
1779                   color=st["color"], marker=st["marker"],
1780                   markersize=st["ms"], lw=st["lw"], ls=st["ls"],
1781                   label=st["label"])
1782         ax_db.plot(lcc[mask], db_k[mask],
1783                   color=st["color"], marker=st["marker"],
1784                   markersize=st["ms"], lw=st["lw"], ls=st["ls"],
1785                   label=st["label"])
1786
1787     ax_x0.set_ylabel(r"$x_0$ (cm)", fontsize=10)
1788     ax_x0.legend(fontsize=8, loc="best")
1789     ax_x0.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1790     ax_x0.tick_params(which="both", direction="in", top=True, right=True)
1791     ax_x0.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1792
1793     ax_db.axhline(0, **ZERO_LINE_KW)
1794     ax_db.set_ylabel(r"$dB_z/dz$ (G/cm)", fontsize=10)
1795     ax_db.set_xlabel("LCC (A)", fontsize=10)
1796     ax_db.legend(fontsize=8, loc="best")
1797     ax_db.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1798     ax_db.tick_params(which="both", direction="in", top=True, right=True)
1799     ax_db.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1800     ax_db.xaxis.set_minor_locator(ticker.AutoMinorLocator())
1801
1802     # — Shared vertical cursor —————

```



```

1803 # One vline per panel (they share x via sharex). On mouse motion the
1804 # cursor snaps to the nearest LCC grid point and the readout box shows
1805 # all values at that LCC.
1806 all_panels = [ax_pol, ax_x0, ax_db]
1807 vl_lines = [ax.axvline(x=np.nan, color="black", lw=1.0, ls="-",
1808                      alpha=0.55, zorder=10)
1809             for ax in all_panels]
1810
1811 readout = ax_pol.text(
1812     0.01, 0.97, "",
1813     transform=ax_pol.transAxes,
1814     fontsize=8, va="top", ha="left",
1815     fontfamily="monospace",
1816     bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
1817             edgecolor="grey", alpha=0.88),
1818     zorder=11
1819 )
1820
1821 def _snap(x_mouse):
1822     """Return the LCC grid point nearest to x_mouse."""
1823     idx = int(np.argmin(np.abs(lcc - x_mouse)))
1824     return idx, lcc[idx]
1825
1826 def _on_move(event):
1827     if event.inaxes not in all_panels or event.xdata is None:
1828         for vl in vl_lines:
1829             vl.set_xdata([np.nan])
1830             readout.set_text("")
1831             fig.canvas.draw_idle()
1832         return
1833
1834     idx, lcc_snap = _snap(event.xdata)
1835     for vl in vl_lines:
1836         vl.set_xdata([lcc_snap])
1837
1838     pol_v = merged["pol_mean"].iloc[idx]
1839     sem_v = merged["pol_sem"].iloc[idx]
1840     row = merged.iloc[idx]
1841
1842     # Build crossing readout lines
1843     crossing_lines = []
1844     for k, label in enumerate(["1", "2", "3"]):
1845         x0_v = row[f"x0_{label}"]
1846         db_v = row[f"dBdz_{label}"]
1847         if not np.isnan(x0_v):
1848             crossing_lines.append(
1849                 f" [{k+1}]   $\epsilon_x$ ={x0_v:.2f} cm  dBdz={db_v:+.3f} G/cm")
1850
1851     lines = [f"LCC = {lcc_snap:.0f} A",
1852             f"Pol = {pol_v:.2f}  $\pm$  {sem_v:.2f} %"] + crossing_lines
1853     readout.set_text("\n".join(lines))
1854     fig.canvas.draw_idle()
1855
1856     fig.canvas.mpl_connect("motion_notify_event", _on_move)
1857
1858     if SAVE_FIGURES:
1859         fname = f"polA_lcc_sweep_ICC1_{POL_ICC1:.3f}.png"
1860         fig.savefig(fname, dpi=130, bbox_inches="tight")
1861         print(f"[plot_pol_lcc_sweep] Saved {fname}")
1862     plt.show()
1863
1864 # =====
1865 # FUNCTION 13 – plot_pol_scatter_matrix (Plot B)
1866 # =====
1867
1868 def plot_pol_scatter_matrix(merged: pd.DataFrame) -> None:
1869     """
1870     Plot B – Scatter matrix: polarization vs  $\epsilon_x$  and vs dBdz.
1871
1872     One row per crossing index (1st / 2nd / 3rd).
1873     Left column : polarization vs  $\epsilon_x$  [cm]
1874     Right column : polarization vs dBdz [G/cm]
1875 
```

```

1876
1877 Points are coloured by LCC (viridis), so you can see both:
1878 • which  $\epsilon_x$  / dBdz value is associated with high polarization
1879 • where on the LCC sweep that occurs
1880
1881 A linear regression line is drawn when  $\geq 5$  points are present,
1882 with the Pearson  $r$  shown in the panel title.
1883 """
1884 from scipy import stats as sp_stats
1885
1886 lcc_vals = merged["LCC"].values
1887 norm = plt.Normalize(lcc_vals.min(), lcc_vals.max())
1888 cmap = plt.cm.viridis
1889
1890 fig, axes = plt.subplots(
1891     3, 2, figsize=(12, 10), layout="constrained"
1892 )
1893 fig.suptitle(
1894     f"Plot B – Polarization vs Crossing Properties\n"
1895     f"colour = LCC (A) | ICC1 = {POL_ICC1:.3f} A | ICC2 = {POL_ICC2:.3g} A",
1896     fontsize=12
1897 )
1898
1899 for k, label in enumerate(["1", "2", "3"]):
1900     st = SERIES_STYLE[k]
1901     pol = merged["pol_mean"].values
1902     sem = merged["pol_sem"].values
1903     x0_k = merged[f"x0_{label}"].values
1904     db_k = merged[f"dBdz_{label}"].values
1905     mask = ~np.isnan(x0_k)
1906
1907     pol_m = pol[mask]; sem_m = sem[mask]
1908     x0_m = x0_k[mask]; db_m = db_k[mask]
1909     lcc_m = lcc_vals[mask]
1910     colors = cmap(norm(lcc_m))
1911
1912     for col_idx, (xdata, xlabel) in enumerate([
1913         (x0_m, r"$x_0$ (cm)"),
1914         (db_m, r"$dB_z/dz$ (G/cm)")]
1915     ):
1916         ax = axes[k, col_idx]
1917
1918         ax.errorbar(xdata, pol_m, yerr=sem_m,
1919                     fmt="none", ecolor="grey",
1920                     elinewidth=0.8, capsize=2, zorder=2)
1921         sc = ax.scatter(xdata, pol_m, c=lcc_m, cmap=cmap,
1922                         norm=norm, s=50, marker=st["marker"],
1923                         edgecolors="k", linewidths=0.4, zorder=3)
1924
1925         # linear regression
1926         if len(xdata) >= 5:
1927             slope, intercept, r, p, _ = sp_stats.linregress(xdata, pol_m)
1928             x_fit = np.linspace(xdata.min(), xdata.max(), 100)
1929             ax.plot(x_fit, slope * x_fit + intercept,
1930                     color=st["color"], lw=1.5, ls="--", alpha=0.8)
1931             p_str = f"p={p:.3f}" if p >= 0.001 else "p<0.001"
1932             ax.set_title(f"{st['label']} | r={r:.3f} {p_str}",
1933                           fontsize=9)
1934         else:
1935             ax.set_title(f"{st['label']} (n={len(xdata)}", fontsize=9)
1936
1937         ax.set_xlabel(xlabel, fontsize=9)
1938         ax.set_ylabel("Polarization (%)", if col_idx == 0 else "",
1939                       fontsize=9)
1940         ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
1941         ax.tick_params(which="both", direction="in",
1942                       top=True, right=True, labelsize=8)
1943         ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
1944         ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
1945
1946         # shared colorbar per row on the right
1947         fig.colorbar(sc, ax=axes[k, :].tolist(),
1948                     label="LCC (A)", shrink=0.7, pad=0.01)

```

```

1949
1950 if SAVE_FIGURES:
1951     fname = f"polB_scatter_matrix_ICC1_{POL_ICC1:.3f}.png"
1952     fig.savefig(fname, dpi=130, bbox_inches="tight")
1953     print(f"[plot_pol_scatter_matrix] Saved {fname}")
1954     plt.show()
1955
1956
1957 # =====
1958 # FUNCTION 14 – plot_pol_phase_space (Plot C)
1959 # =====
1960
1961 def plot_pol_phase_space(merged: pd.DataFrame,
1962                         sona: dict = None) -> None:
1963     """
1964     Plot C – Phase-space map:  $x$  vs dBdz, coloured by polarization.
1965
1966     Each point is one (LCC, crossing-index) pair.
1967     x-axis : zero-crossing position  $x$  [cm]
1968     y-axis : field gradient dBdz [G/cm]
1969     colour : beam polarization [%] (hot colormap)
1970     marker : crossing index (circle / square / diamond)
1971     size : proportional to LCC so the trajectory through ( $x$ , dBdz)
1972           space as LCC increases is visible
1973
1974     A horizontal dashed line at dBdz = 0 separates positive and negative
1975     gradients. The Sona threshold is drawn if sona is provided.
1976
1977     This is the most direct answer to "which ( $x$ , dBdz) combination gives
1978     the highest polarization" – look for the bright yellow region.
1979     """
1980     pol_min = merged["pol_mean"].min()
1981     pol_max = merged["pol_mean"].max()
1982     norm_pol = plt.Normalize(pol_min, pol_max)
1983     cmap_pol = plt.cm.RdYlGn # red = low pol, green = high pol
1984
1985     lcc_vals = merged["LCC"].values
1986     lcc_min, lcc_max = lcc_vals.min(), lcc_vals.max()
1987
1988     fig, ax = plt.subplots(figsize=(10, 7), layout="constrained")
1989     fig.suptitle(
1990         f"Plot C – Phase-space  $x$  vs  $dB_z/dz$ , colour = polarization\n"
1991         f"ICC1 = {POL_ICC1:.3f} A | ICC2 = {POL_ICC2:.3g} A | "
1992         f"marker = crossing index | size  $\propto$  LCC",
1993         fontsize=11
1994     )
1995
1996     all_sc = None # keep last scatter for colorbar
1997
1998     for k, label in enumerate(["1", "2", "3"]):
1999         st = SERIES_STYLE[k]
2000         x0_k = merged[f"x0_{label}"].values
2001         db_k = merged[f"dBdz_{label}"].values
2002         pol = merged["pol_mean"].values
2003         mask = ~np.isnan(x0_k)
2004
2005         x0_m = x0_k[mask]; db_m = db_k[mask]
2006         pol_m = pol[mask]; lcc_m = lcc_vals[mask]
2007
2008         # size: scale LCC linearly from 40 to 160 pt2
2009         sizes = 40 + 120 * (lcc_m - lcc_min) / max(lcc_max - lcc_min, 1)
2010
2011         sc = ax.scatter(x0_m, db_m,
2012                        c=pol_m, cmap=cmap_pol, norm=norm_pol,
2013                        s=sizes, marker=st["marker"],
2014                        edgecolors="k", linewidths=0.5,
2015                        zorder=4, label=st["label"])
2016         all_sc = sc
2017
2018         # annotate extreme LCC values for orientation
2019         if len(lcc_m):
2020             for idx in [0, -1]:
2021                 ax.annotate(f"{lcc_m[idx]:.0f}A",

```

```

2022                 xy=(x0_m[idx], db_m[idx]),
2023                 fontsize=6, color="dimgrey",
2024                 xytext=(4, 4), textcoords="offset points")
2025
2026 # reference lines
2027 ax.axhline(0, **ZERO_LINE_KW)
2028 if sona is not None:
2029     thr = sona["dBdz_sona_Gcm"]
2030     ax.axhline(thr, color="crimson", lw=1.2, ls="--", alpha=0.8,
2031               label=f"±{thr:.1f} G/cm (Sona)")
2032     ax.axhline(-thr, color="crimson", lw=1.2, ls="--", alpha=0.8)
2033
2034 ax.set_xlabel(r"Zero-crossing position $x_0$ (cm)", fontsize=11)
2035 ax.set_ylabel(r"Gradient $dB_z/dz$ (G/cm)", fontsize=11)
2036 ax.legend(fontsize=9, loc="upper left", title="Crossing index")
2037 ax.tick_params(which="both", direction="in", top=True, right=True)
2038 ax.xaxis.set_minor_locator(ticker.AutoMinorLocator())
2039 ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
2040 ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
2041
2042 if all_sc is not None:
2043     cbar = fig.colorbar(all_sc, ax=ax, label="Beam polarization (%)",
2044                       shrink=0.85)
2045     cbar.ax.tick_params(labelsize=9)
2046
2047 # size legend
2048 for lcc_ref in [80, 120, 160]:
2049     s_ref = 40 + 120 * (lcc_ref - lcc_min) / max(lcc_max - lcc_min, 1)
2050     ax.scatter([], [], s=s_ref, color="grey", edgecolors="k",
2051               linewidths=0.5, label=f"LCC={lcc_ref} A")
2052 ax.legend(fontsize=8, loc="upper left",
2053           title="Crossing / LCC size", title_fontsize=8)
2054
2055 if SAVE_FIGURES:
2056     fname = f"polC_phase_space_ICC1_{POL_ICC1:.3f}.png"
2057     fig.savefig(fname, dpi=130, bbox_inches="tight")
2058     print(f"[plot_pol_phase_space] Saved {fname}")
2059 plt.show()
2060
2061 # =====
2062 # FUNCTION 15 – plot_br_summary_grid
2063 # =====
2064
2065 def plot_br_summary_grid(scan: pd.DataFrame) -> None:
2066     """
2067     Summary grid of radial field Br at each zero-crossing vs LCC.
2068
2069     Layout
2070     -----
2071     Rows      = ICC1 values    (one row per ICC1, same as plot_summary_grid)
2072     Columns   = beam radii     (one column per r in BR_RADII_CM)
2073
2074     Within each panel – one line series per crossing index:
2075     1st crossing – steelblue circle ●
2076     2nd crossing – orange square ■
2077     3rd crossing – seagreen diamond ◆
2078     (Uses SERIES_STYLE colours, consistent with all other summary figures.)
2079
2080     Y-axis: Br [G] = -(dBz/dz) * r / 2 at the crossing position.
2081
2082     Because the three radii are just a multiplicative rescaling of dBdz
2083     (Br = dBdz * r / 2), the shape of every curve is identical across
2084     columns – only the scale changes. This lets you immediately read off
2085     what radial field amplitude a beam of a given size experiences at each
2086     Sona reversal.
2087
2088     Parameters
2089     -----
2090     scan : pd.DataFrame – output of scan_conditions()
2091     """
2092     icc1_vals = sorted(scan["ICC1"].unique())
2093     lcc_vals = sorted(scan["LCC"].unique())

```

```

2095 n_icc1 = len(icc1_vals)
2096 n_r = len(BR_RADII_CM)
2097 icc2_val = scan["ICC2"].iloc[0]
2098
2099 # — Pre-compute Br at every crossing for every condition —————
2100 # scan["all_crossings"] holds (x0, dBdz) tuples.
2101 # Br_k at crossing k = -dBdz_k * r / 2
2102 max_crossings = max(len(r["all_crossings"]) for _, r in scan.iterrows())
2103
2104 # crossing_data[icc1_idx][crossing_k] = [(lcc, br_r0, br_r1, br_r2), ...]
2105 crossing_data = []
2106 for icc1 in icc1_vals:
2107     sub = scan[np.abs(scan["ICC1"] - icc1) < 1e-6].sort_values("LCC")
2108     per_k = [[] for _ in range(max_crossings)]
2109     for _, row in sub.iterrows():
2110         for k, (x0, dBdz) in enumerate(row["all_crossings"]):
2111             br_vals = [-dBdz * r / 2 for r in BR_RADII_CM]
2112             per_k[k].append((row["LCC"],) + tuple(br_vals))
2113     crossing_data.append(per_k)
2114
2115 # — Figure —————
2116 fig, axes = plt.subplots(
2117     n_icc1, n_r,
2118     figsize=(4.5 * n_r, 3.5 * n_icc1),
2119     sharex=True,
2120     layout="constrained",
2121     gridspec_kw={"hspace": 0.10, "wspace": 0.20}
2122 )
2123 axes = np.atleast_2d(axes)
2124
2125 fig.suptitle(
2126     f"Radial Field $B_r$ at Zero-Crossings - ICC2 = {icc2_val:.3g} A\n"
2127     f"$B_r = -\frac{dB_z}{dz} \cdot \frac{r}{2}$"
2128     f"| rows = ICC1 | columns = beam radius r [cm] | "
2129     f"colour = crossing index",
2130     fontsize=12, y=1.01
2131 )
2132
2133 for row_idx, icc1 in enumerate(icc1_vals):
2134     per_k = crossing_data[row_idx]
2135
2136     for col_idx, r in enumerate(BR_RADII_CM):
2137         ax = axes[row_idx, col_idx]
2138         r_col_idx = col_idx # index into the br_vals tuple (offset by 1 for lcc)
2139         legend_handles = []
2140
2141         for k, pts in enumerate(per_k):
2142             if not pts:
2143                 continue
2144             style = SERIES_STYLE[min(k, len(SERIES_STYLE) - 1)]
2145             lcc_k = [p[0] for p in pts]
2146             br_k = [p[1 + r_col_idx] for p in pts]
2147
2148             h, = ax.plot(lcc_k, br_k,
2149                         color=style["color"], marker=style["marker"],
2150                         markersize=style["ms"], linewidth=style["lw"],
2151                         linestyle=style["ls"], zorder=3 + k,
2152                         label=style["label"])
2153             legend_handles.append(h)
2154
2155         ax.axhline(0, **ZERO_LINE_KW)
2156         ax.tick_params(which="both", direction="in", top=True, right=True)
2157         ax.yaxis.set_minor_locator(ticker.AutoMinorLocator())
2158         ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
2159
2160         # Row label on the leftmost column only
2161         if col_idx == 0:
2162             ax.set_ylabel(f"ICC1 = {icc1:.3f} A\n$B_r$ (G)", fontsize=9)
2163         else:
2164             ax.set_ylabel(r"$B_r$ (G)", fontsize=9)
2165
2166         # Column title on the top row only
2167         if row_idx == 0:

```

```

2168         ax.set_title(f"r = {r:.1f} cm", fontsize=11, pad=6)
2169
2170         # Legend on the top-left panel only to save space
2171         if row_idx == 0 and col_idx == 0 and legend_handles:
2172             ax.legend(handles=legend_handles, fontsize=7, loc="best",
2173                       title="crossing", title_fontsize=7)
2174
2175     axes[-1, n_r // 2].set_xlabel("LCC (A)", fontsize=10)
2176     for col_idx in range(n_r):
2177         if len(lcc_vals) > 20:
2178             axes[-1, col_idx].set_xticks(lcc_vals[::4])
2179         else:
2180             axes[-1, col_idx].set_xticks(lcc_vals)
2181
2182     if SAVE_FIGURES:
2183         fname = f"br_summary_grid_ICC2_{icc2_val:.3g}.png"
2184         fig.savefig(fname, dpi=130, bbox_inches="tight")
2185         print(f"[plot_br_summary_grid] Saved {fname}")
2186     plt.show()
2187
2188 # =====
2189 # FUNCTION 16 – compute_sona_freq
2190 # =====
2191
2192 def compute_sona_freq(df: pd.DataFrame,
2193                      icc1: float = POL_ICC1) -> pd.DataFrame:
2194     """
2195     Compute the effective Sona oscillation frequency and transition energies
2196     for every LCC value at a given ICC1, following Kannis et al. (2022).
2197
2198     Physics
2199     -----
2200     As the beam travels through the Sona unit at velocity  $v_{\text{atom}}$ , it
2201     experiences  $B_r(z) = -(dB_z/dz) \cdot r/2$  as a time-varying field. This
2202     oscillates once between the two inflection points of  $B_z(z)$  – i.e. the
2203     two zeros of  $dB_z/dz$  that flank the field reversal. Their separation
2204     defines the spatial wavelength  $\lambda$ . The effective RF frequency seen by
2205     the atom is:
2206
2207          $f = v_{\text{atom}} / \lambda$ 
2208
2209     and can drive hyperfine transitions with energy differences:
2210
2211          $\Delta E_n = n \cdot h \cdot f \quad (n = 1, 2, 3, \dots)$ 
2212
2213     Parameters
2214     -----
2215     df : output of load_data()
2216     icc1 : ICC1 value to analyse [A]
2217
2218     Returns
2219     -----
2220     pd.DataFrame with columns:
2221         LCC [A]
2222         lambda_cm [cm] spatial wavelength of Br oscillation
2223         f_MHz [MHz] effective RF frequency
2224         dE1_neV [neV]  $\Delta E$  for n=1
2225         dE2_neV [neV]  $\Delta E$  for n=2
2226         ...up to dE{SONA_N_MAX}_neV
2227         B_max_G [G] peak positive Bz (proxy for coil field strength)
2228         br_zero_lo [cm] lower (beam-direction) Br zero position
2229         br_zero_hi [cm] upper (beam-direction) Br zero position
2230         n_br_zeros number of Br zeros found in search window
2231
2232     """
2233     m_p = 1.6726219e-27 # kg
2234     e_C = 1.60218e-19 # C
2235     h_eV = 4.135667696e-15 # eV·s
2236     E_J = SONA_BEAM_ENERGY_KEV * 1e3 * e_C
2237     v = np.sqrt(2.0 * E_J / m_p)
2238
2239     records = []
2240

```

```

2241 for lcc in sorted(df[COL_LCC].unique()):
2242     mask = ((np.abs(df[COL_LCC] - lcc) < 1e-6) &
2243             (np.abs(df[COL_ICC1] - icc1) < 1e-6))
2244     prof = df[mask].sort_values(COL_POS).reset_index(drop=True)
2245     if prof.empty:
2246         continue
2247
2248     pos = prof[COL_POS].values
2249     fld = prof[COL_FIELD].values
2250     mid, dBdz = field_derivative(pos, fld)
2251
2252     # Peak positive field on the high-z (entrance) side
2253     hi_mask = pos > SONA_BR_SEARCH_ZMAX
2254     B_max = fld[hi_mask].max() if hi_mask.any() else np.nan
2255
2256     # Find zeros of dBdz (= zeros of Br) in the reversal window
2257     win = (mid >= SONA_BR_SEARCH_ZMIN) & (mid <= SONA_BR_SEARCH_ZMAX)
2258     mid_w = mid[win]; dBdz_w = dBdz[win]
2259
2260     br_zeros = []
2261     for i in range(len(dBdz_w) - 1):
2262         if dBdz_w[i] * dBdz_w[i + 1] < 0:
2263             z0 = (mid_w[i]
2264                  + (mid_w[i+1] - mid_w[i])
2265                  * (-dBdz_w[i]) / (dBdz_w[i+1] - dBdz_w[i]))
2266             br_zeros.append(float(z0))
2267         elif dBdz_w[i] == 0.0: # exact zero (catches -0.0, IEEE-754)
2268             # Only count if neighbours have opposite signs (true zero crossing)
2269             if i > 0 and dBdz_w[i - 1] * dBdz_w[i + 1] < 0:
2270                 br_zeros.append(float(mid_w[i]))
2271
2272     rec = dict(LCC=lcc, B_max_G=B_max, n_br_zeros=len(br_zeros),
2273               lambda_cm=np.nan, f_MHz=np.nan,
2274               br_zero_lo=np.nan, br_zero_hi=np.nan)
2275
2276     if len(br_zeros) >= 2:
2277         lam_cm = abs(br_zeros[-1] - br_zeros[0])
2278         f_Hz = v / (lam_cm * 1e-2)
2279         rec["lambda_cm"] = round(lam_cm, 4)
2280         rec["f_MHz"] = round(f_Hz / 1e6, 6)
2281         rec["br_zero_lo"] = min(br_zeros)
2282         rec["br_zero_hi"] = max(br_zeros)
2283         for n in range(1, SONA_N_MAX + 1):
2284             dE_neV = n * h_eV * f_Hz * 1e9 # neV
2285             rec[f"dE{n}_neV"] = round(dE_neV, 4)
2286
2287     records.append(rec)
2288
2289     result = pd.DataFrame(records).sort_values("LCC").reset_index(drop=True)
2290
2291     # Fill missing dE columns with NaN if any rows had < 2 Br zeros
2292     for n in range(1, SONA_N_MAX + 1):
2293         col = f"dE{n}_neV"
2294         if col not in result.columns:
2295             result[col] = np.nan
2296
2297     print(f"[compute_sona_freq] {len(result)} LCC points | ICC1={icc1:.3f} "
2298           f"| v={v:.3e} m/s (E={SONA_BEAM_ENERGY_KEV} keV)")
2299     print(f"  λ range: {result['lambda_cm'].min():.2f} - "
2300           f"{result['lambda_cm'].max():.2f} cm")
2301     print(f"  f range: {result['f_MHz'].min():.3f} - "
2302           f"{result['f_MHz'].max():.3f} MHz")
2303     print(f"  ΔE(n=1): {result['dE1_neV'].min():.2f} - "
2304           f"{result['dE1_neV'].max():.2f} neV")
2305     return result
2306
2307
2308 # =====
2309 # FUNCTION 17 – plot_sona_freq
2310 # =====
2311
2312 def plot_sona_freq(sona_freq: pd.DataFrame,
2313                   df_pol: pd.DataFrame = None) -> None:

```



```

2314 """
2315 Four-panel figure showing the Sona frequency analysis vs LCC.
2316
2317 Panel 1 – Beam polarization [%] vs LCC (if df_pol provided)
2318 with ±1 std shading and ±1 SEM errorbars.
2319
2320 Panel 2 – Spatial wavelength  $\lambda$  [cm] vs LCC.
2321  $\lambda$  grows as LCC increases because the field reversal stretches.
2322
2323 Panel 3 – Effective RF frequency  $f$  [MHz] vs LCC.
2324  $f = v_{\text{atom}} / \lambda$  decreases as  $\lambda$  grows.
2325
2326 Panel 4 – Transition energies  $\Delta E_n = n \cdot h \cdot f$  [neV] vs LCC,
2327 for  $n = 1 \dots \text{SONA\_N\_MAX}$ . Each  $n$  is a separate coloured line.
2328 Horizontal dashed lines mark known Breit-Rabi reference
2329 energies from BREIT_RABI_REFS.
2330
2331 All panels share the same x-axis (LCC). A shared vertical cursor snaps
2332 to the nearest LCC grid point and shows a readout box with all values.
2333
2334 Parameters
2335 -----
2336 sona_freq : output of compute_sona_freq()
2337 df_pol : output of load_polarization() – optional; if None, panel 1
2338 is replaced by a placeholder.
2339 """
2340 lcc = sona_freq["LCC"].values
2341 has_pol = df_pol is not None
2342
2343 # — Merge polarization onto the magnetic scan LCC grid
2344 pol_mean = np.full(len(lcc), np.nan)
2345 pol_sem = np.full(len(lcc), np.nan)
2346 pol_std = np.full(len(lcc), np.nan)
2347 if has_pol:
2348     for i, lv in enumerate(lcc):
2349         row = df_pol[np.abs(df_pol["LCC"] - lv) < 0.5]
2350         if not row.empty:
2351             pol_mean[i] = row.iloc[0]["pol_mean"]
2352             pol_sem[i] = row.iloc[0]["pol_sem"]
2353             pol_std[i] = row.iloc[0]["pol_std"]
2354
2355 # — Colour palette for  $n = 1 \dots N_{\text{MAX}}$ 
2356 n_colors = plt.cm.plasma(np.linspace(0.1, 0.85, SONA_N_MAX))
2357
2358 # — Figure
2359 n_panels = 4
2360 height_ratios = [1.4, 0.9, 0.9, 1.4]
2361 fig, axes = plt.subplots(
2362     n_panels, 1, figsize=(12, 10),
2363     sharex=True, layout="constrained",
2364     gridspec_kw={"height_ratios": height_ratios, "hspace": 0.05}
2365 )
2366 ax_pol, ax_lam, ax_f, ax_dE = axes
2367
2368 fig.suptitle(
2369     f"Sona Frequency Analysis – ICC1 = {POL_ICC1:.3f} A | "
2370     f"ICC2 = {POL_ICC2:.3g} A | "
2371     f"$E_{\{\text{beam}\}} = \{\text{SONA\_BEAM\_ENERGY\_KEV}:\text{.1f}\} \text{ keV}\n"
2372     f"$\lambda$ = distance between $B_r$ zeros, "
2373     f"$f = v_{\text{atom}} / \lambda$, "
2374     f"$\Delta E_n = n \cdot h \cdot f$",
2375     fontsize=11
2376 )
2377
2378 # — Panel 1: polarization
2379 if has_pol:
2380     valid = ~np.isnan(pol_mean)
2381     ax_pol.fill_between(lcc[valid],
2382                        pol_mean[valid] - pol_std[valid],
2383                        pol_mean[valid] + pol_std[valid],
2384                        color="steelblue", alpha=0.15, label="±1 std")
2385     ax_pol.errorbar(lcc[valid], pol_mean[valid], yerr=pol_sem[valid],
2386                    fmt="o-", color="steelblue", markersize=3.5,

```

```

2387         lw=1.4, elinewidth=0.9, capsize=2.5,
2388         label="mean ± SEM", zorder=4)
2389     ax_pol.set_ylabel("Polarization (%)", fontsize=10)
2390     ax_pol.legend(fontsize=8, loc="upper left")
2391     else:
2392         ax_pol.text(0.5, 0.5, "no polarization data",
2393                    transform=ax_pol.transAxes, ha="center", va="center",
2394                    color="grey", fontsize=10)
2395     ax_pol.set_ylabel("Polarization (%)", fontsize=10)
2396
2397     # — Panel 2:  $\lambda$  —————
2398     valid_lam = ~np.isnan(sona_freq["lambda_cm"])
2399     ax_lam.plot(lcc[valid_lam], sona_freq["lambda_cm"][valid_lam],
2400               "o-", color="seagreen", markersize=3.5, lw=1.5)
2401     ax_lam.set_ylabel(r"$\lambda$ (cm)", fontsize=10)
2402     ax_lam.yaxis.set_minor_locator(ticker.AutoMinorLocator())
2403
2404     # — Panel 3:  $f$  —————
2405     valid_f = ~np.isnan(sona_freq["f_MHz"])
2406     ax_f.plot(lcc[valid_f], sona_freq["f_MHz"][valid_f],
2407             "o-", color="darkorange", markersize=3.5, lw=1.5)
2408     ax_f.set_ylabel("$f$ (MHz)", fontsize=10)
2409     ax_f.yaxis.set_minor_locator(ticker.AutoMinorLocator())
2410
2411     # — Panel 4:  $\Delta E_n$  —————
2412     for n in range(1, SONA_N_MAX + 1):
2413         col = f"dE{n}_neV"
2414         if col not in sona_freq.columns:
2415             continue
2416         valid_n = ~np.isnan(sona_freq[col])
2417         ax_dE.plot(lcc[valid_n], sona_freq[col][valid_n],
2418                 "o-", color=n_colors[n - 1], markersize=3.5, lw=1.5,
2419                 label=f"$n = {n}$")
2420
2421     # Breit-Rabi reference lines
2422     for e_ref, lbl in BREIT_RABI_REFS:
2423         ax_dE.axhline(e_ref, color="crimson", lw=1.2, ls="--", alpha=0.85,
2424                     label=lbl)
2425
2426     ax_dE.set_ylabel(r"$\Delta E_n = n \cdot h \cdot f$ (neV)", fontsize=10)
2427     ax_dE.set_xlabel("LCC (A)", fontsize=10)
2428     ax_dE.legend(fontsize=8, loc="upper right", ncol=3,
2429                 title="n (harmonic)", title_fontsize=8)
2430     ax_dE.yaxis.set_minor_locator(ticker.AutoMinorLocator())
2431
2432     for ax in axes:
2433         ax.tick_params(which="both", direction="in", top=True, right=True)
2434         ax.grid(True, which="major", ls=":", lw=0.5, alpha=0.7)
2435         ax.grid(True, which="minor", ls=":", lw=0.2, alpha=0.35)
2436     axes[-1].xaxis.set_minor_locator(ticker.AutoMinorLocator())
2437
2438     # — Shared vertical cursor —————
2439     vl_lines = [ax.axvline(x=np.nan, color="black", lw=1.0, ls="-",
2440                          alpha=0.55, zorder=10)
2441                for ax in axes]
2442
2443     readout = ax_pol.text(
2444         0.99, 0.97, "",
2445         transform=ax_pol.transAxes,
2446         fontsize=8, va="top", ha="right",
2447         fontfamily="monospace",
2448         bbox=dict(boxstyle="round,pad=0.4", facecolor="white",
2449                 edgcolor="grey", alpha=0.88),
2450         zorder=11
2451     )
2452
2453     def _snap(x_mouse):
2454         idx = int(np.argmin(np.abs(lcc - x_mouse)))
2455         return idx, lcc[idx]
2456
2457     def _on_move(event):
2458         if event.inaxes not in axes or event.xdata is None:
2459             for vl in vl_lines: vl.set_xdata([np.nan])

```

```

2460         readout.set_text("")
2461         fig.canvas.draw_idle()
2462         return
2463
2464     idx, lcc_snap = _snap(event.xdata)
2465     for vl in vlines: vl.set_xdata([lcc_snap])
2466
2467     row = sona_freq[np.abs(sona_freq["LCC"] - lcc_snap) < 0.5]
2468     lam = row.iloc[0]["lambda_cm"] if not row.empty else np.nan
2469     f = row.iloc[0]["f_MHz"] if not row.empty else np.nan
2470     dE1 = row.iloc[0]["dE1_neV"] if not row.empty else np.nan
2471
2472     pol_str = (f"{pol_mean[idx]:.1f} ± {pol_sem[idx]:.1f} %"
2473               if not np.isnan(pol_mean[idx]) else "-")
2474
2475     lines = [f"LCC = {lcc_snap:.0f} A",
2476             f"Pol = {pol_str}",
2477             f"λ = {lam:.2f} cm" if not np.isnan(lam) else "λ = -",
2478             f"f = {f:.3f} MHz" if not np.isnan(f) else "f = -",
2479             f"Δ1E = {dE1:.2f} neV" if not np.isnan(dE1) else "Δ1E = -"]
2480     # Add all n harmonics
2481     for n in range(2, SONA_N_MAX + 1):
2482         col = f"dE{n}_neV"
2483         val = row.iloc[0][col] if (not row.empty and col in row.columns) else np.nan
2484         if not np.isnan(val):
2485             lines.append(f"ΔE{n} = {val:.2f} neV")
2486
2487     readout.set_text("\n".join(lines))
2488     fig.canvas.draw_idle()
2489
2490     fig.canvas.mpl_connect("motion_notify_event", _on_move)
2491
2492     if SAVE_FIGURES:
2493         fname = f"sona_freq_ICC1_{POL_ICC1:.3f}.png"
2494         fig.savefig(fname, dpi=130, bbox_inches="tight")
2495         print(f"[plot_sona_freq] Saved {fname}")
2496     plt.show()
2497
2498     # =====
2499     # MAIN – entry point
2500     # =====
2501     if __name__ == "__main__":
2502
2503         # 1. Sona parameters (used for threshold line in summary grid)
2504         sona = compute_sona_parameters(BEAM_ENERGY_KEV, BEAM_RADIUS_CM)
2505
2506         # 2. Load data – clean, remove ICC2=0, keep only ICC2_SELECT plane
2507         df = load_data(DATA_PATH, icc2_select=ICC2_SELECT)
2508
2509         # 3. Scan all (LCC, ICC1) combinations
2510         scan = scan_conditions(df)
2511
2512         # 4. Print summary table
2513         print("\n— Zero-crossing summary —")
2514         print(scan[["LCC", "ICC1", "ICC2",
2515                   "n_crossings", "x0_primary",
2516                   "dBdz_primary"]].to_string(index=False,
2517                                             float_format="{:.3f}".format))
2518
2519         # 5 & 6. Fig 1a and Fig 1b – built first, linked, then shown together.
2520         #
2521         # Both figures must exist before plt.show() is called on either,
2522         # otherwise the first show() blocks and the second figure is never
2523         # created (and therefore never linked).
2524         #
2525         # We temporarily monkey-patch plt.show to a no-op while building the
2526         # figures, then restore it, link the axes, and call show() once.
2527         _real_show = plt.show
2528         plt.show = lambda *a, **kw: None # suppress internal show calls
2529
2530         axes_1a = plot_profiles_grid(df, scan=scan)
2531         axes_1b = plot_gradients_grid(df, scan=scan, sona=sona)
2532

```

```

2533 plt.show = _real_show                                # restore
2534
2535 link_fig1_axes(axes_1a, axes_1b)                     # cross-figure x-axis linking
2536 plt.show()                                           # display both figures together
2537
2538 # 7. Main output: summary grid (rows=ICCl, cols=x0 / dBdz vs LCC)
2539 plot_summary_grid(scan, sona=sona)
2540
2541 # 8. Beam polarization vs LCC (simple overview)
2542 df_pol = load_polarization(POL_DATA_PATH)
2543 plot_polarization(df_pol)
2544
2545 # 9. Correlation plots A, B, C
2546 merged = merge_pol_scan(df_pol, scan, iccl=POL_ICC1)
2547 plot_pol_lcc_sweep(merged)                           # Plot A: 3-panel LCC sweep
2548 plot_pol_scatter_matrix(merged)                      # Plot B: scatter pol vs x0/dBdz
2549 plot_pol_phase_space(merged, sona)                  # Plot C: phase-space map
2550
2551 # 10. Interactive profile inspector – Bz + dBdz (vertical cursor + sliders)
2552 # Requires an interactive backend (Qt5 in Spyder:
2553 # Tools → Preferences → IPython console → Graphics → Qt5)
2554 plot_profile_interactive(df, lcc=DEFAULT_LCC, iccl=DEFAULT_ICC1)
2555
2556 # 11. Interactive Bz + Br figure (three radii: 0.5, 1.0, 1.5 cm)
2557 plot_br_interactive(df, lcc=DEFAULT_LCC, iccl=DEFAULT_ICC1)
2558
2559 # 12. Br at zero-crossings summary grid (rows=ICCl, cols=radius)
2560 plot_br_summary_grid(scan)
2561
2562 # 13. Sona frequency analysis  $\lambda$ (, f,  $\Delta E_n$  vs LCC) – Kannis et al. framework
2563 sona_freq = compute_sona_freq(df, iccl=POL_ICC1)
2564 plot_sona_freq(sona_freq, df_pol=df_pol)
2565

```

Listing 5: MagScan_Ana4.py — field map processing and Sona parameter analysis (current status)

References

- [1] C. Kannis, *Theoretical and experimental investigation of Sona transitions*, Ph.D. thesis, RWTH Aachen University (2023).
- [2] R. Engels et al., *New Insights into Sona Transitions*, (2024).
- [3] A. Kponou et al., *Sona Transition Studies in the RHIC OPPIS*, Proc. PSTP2007 / AIP Conf. Proc.
- [4] E. P. Antishev and A. S. Belov, *Study of Adiabaticity of Hydrogen Atoms Spin Motion in Changing Magnetic Fields*, AIP Conf. Proc. **980**, 263 (2008), Proc. PSTP2007.
- [5] C. S. Kannis et al., *New Application of a Sona Transition Unit: Observation of Direct Transitions Between Quantum States With Energy Differences of 10 neV and Below*, JPS Conf. Proc. **37**, 021209 (2022), Proc. 24th Int. Spin Symposium (SPIN2021).
- [6] N. Kolachevsky, A. Matveev, J. Alnis, C. G. Parthey, S. G. Karshenboim, and T. W. Hänsch, *Measurement of the 2S Hyperfine Interval in Atomic Hydrogen*, Phys. Rev. Lett. **102**, 213002 (2009).